

Contents

1	Abschlussprüfung Sommer 2026	4
1.1	Certified IT Business Manager (IHK)	4
1.1.1	Projektarbeit im Rahmen der Prüfung zum Certified IT Business Manager (IHK)	4
1.2	SPERRVERMERK	5
1.3	VORWORT	5
1.3.1	I. Einleitung	5
1.3.2	II. Projektumfeld	5
1.3.3	III. Meine Tätigkeiten im Unternehmen	6
1.3.4	IV. Allgemeine Informationen	6
1.4	INHALTSVERZEICHNIS	6
1.5	ABBILDUNGSVERZEICHNIS	8
1.6	TABELLENVERZEICHNIS	9
1.7	ABKÜRZUNGSVERZEICHNIS	9
2	1 Projektinitiierung	11
2.1	1.1 Projektumfeld	11
2.2	1.2 Ausgangssituation	12
2.3	1.3 Ist-Analyse	12
2.4	1.4 SWOT-Analyse	13
2.5	1.5 Problemstellung	13
2.6	1.6 Rahmenbedingungen	14
2.6.1	1.6.1 Zeitliche Rahmenbedingungen	14
2.6.2	1.6.2 Finanzielle Rahmenbedingungen	14
2.6.3	1.6.3 Technische Rahmenbedingungen	15
2.6.4	1.6.4 Organisatorische Rahmenbedingungen	15
2.6.5	1.6.5 Rechtliche Rahmenbedingungen	16
2.7	1.7 Soll-Konzept	16
2.8	1.8 Projektziele nach SMART	17
2.9	1.9 Projektbegründung	17
2.10	1.10 Projektabgrenzung	17
2.11	1.11 Projektschnittstellen	18
2.12	1.12 Projektauftrag	18
3	2 Projektplanung	18
3.1	2.1 Vorgehensmodell	18
3.2	2.2 Projektphasen	19
3.3	2.3 Projektstrukturplan (PSP)	19
3.4	2.4 Arbeitspakete	20

3.5	2.5 Meilensteinplanung	20
3.6	2.6 Ressourcenplanung	21
3.7	2.7 Kostenplanung	21
3.8	2.8 Kommunikationsplanung	22
3.9	2.9 Risikoanalyse	22
3.10	2.10 Qualitätsplanung	23
3.11	2.11 Abweichungen vom Projektantrag	23
4	3 Analyse und Konzeption	24
4.1	3.1 Anforderungsanalyse	24
4.2	3.2 Lastenheft / Fachkonzept	24
4.3	3.3 Stakeholderanalyse	25
4.4	3.4 Make-or-Buy-Entscheidung	25
4.5	3.5 Machbarkeitsanalyse	26
4.6	3.6 Wirtschaftlichkeitsanalyse	27
4.7	3.7 Nutzwertanalyse	27
4.8	3.8 Zero-Trust-Konzept	28
4.9	3.9 RBAC-Modell	28
4.10	3.10 Lenkungsausschuss	29
4.11	3.11 Rolle des Projektleiters	29
4.12	3.12 Datenschutz- und Sicherheitskonzept	29
5	4 Technischer Entwurf	30
5.1	4.1 Zielplattform	30
5.2	4.2 Architekturdesign	30
5.3	4.3 GitHub-Workflow-Integration	30
5.4	4.4 Self-Service-Prozess	31
5.5	4.5 Datenmodell	31
5.6	4.6 Geschäftslogik	32
5.7	4.7 Audit-Logging	32
5.8	4.8 Schnittstellen	32
5.9	4.9 Maßnahmen zur Qualitätssicherung	32
5.10	4.10 Pflichtenheft / Datenverarbeitungskonzept	33
6	5 Umsetzung	33
6.1	5.1 Aufbau der Entwicklungsumgebung	33
6.2	5.2 Implementierung der Datenstrukturen	33
6.3	5.3 Implementierung des RBAC-Modells	34
6.4	5.4 Implementierung der Benutzeroberfläche	34
6.5	5.5 Implementierung der GitHub-Automatisierung	34
6.6	5.6 Implementierung der Geschäftslogik	35

6.7	5.7 Implementierung der Audit-Protokollierung	35
6.8	5.8 Implementierung der Qualitätssicherung	35
6.9	5.9 Entwicklerdokumentation	35
7	6 Test und Abnahme	36
7.1	6.1 Testkonzept	36
7.2	6.2 Funktionstests	36
7.3	6.3 Integrationstests	37
7.4	6.4 Security-Tests	37
7.5	6.5 Datenschutzprüfung	37
7.6	6.6 Testfallmatrix	37
7.7	6.7 Fehleranalyse	37
7.8	6.8 Soll-Ist-Vergleich	38
7.9	6.9 Abnahme	39
8	7 Einführung und Dokumentation	39
8.1	7.1 Einführungskonzept	39
8.2	7.2 Pilotbetrieb	40
8.3	7.3 Schulung	40
8.4	7.4 Change Management	40
8.5	7.5 Benutzerdokumentation	40
8.6	7.6 Betriebsdokumentation	40
8.7	7.7 Übergabe	40
9	8 Projektabschluss	41
9.1	8.1 Projektergebnis	41
9.2	8.2 Soll-Ist-Vergleich (wirtschaftlich)	41
9.3	8.3 Wirtschaftliche Bewertung	41
9.4	8.4 Lessons Learned	41
9.5	8.5 Risiken nach Projektabschluss	42
9.6	8.6 Ausblick	42
9.7	8.7 Persönliches Fazit	42
10	LITERATURVERZEICHNIS	43
10.1	Normen & Standards	43
10.2	Fachliteratur	43
10.3	Projektmanagement	43
10.4	IT-Security & Compliance	43
11	EIDESSTATTLICHE ERKLÄRUNG	44
11.1	ANHANG	44
11.1.1	A1 Detaillierte Zeitplanung	44

11.1.2	A2 Lastenheft-Auszug	47
11.1.3	A3 Use-Case-Diagramm	49
11.1.4	A4 Pflichtenheft-Auszug	49
11.1.5	A5 Datenmodell	51
11.1.6	A6 EPK-Prozessbeschreibung	51
11.1.7	A7 Oberflächenentwürfe (Wireframes)	53
11.1.8	A8 Screenshots der Anwendung	54
11.1.9	A9 Entwicklerdokumentation	54
11.1.10	A10 Testfall Konsole	55
11.1.11	A11 Schnittstellenübersicht	55
11.1.12	A12 Klassendiagramm	56
11.1.13	A13 Benutzerdokumentation	57
11.1.14	A14 Datenschutz-Checkliste	58
11.1.15	A15 Abnahmeprotokoll	59

1 Abschlussprüfung Sommer 2026

1.1 Certified IT Business Manager (IHK)

1.1.1 Projektarbeit im Rahmen der Prüfung zum Certified IT Business Manager (IHK)

Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration

Abgabedatum: Stuttgart, den 01.11.2026

Prüfungsbewerber: Daniel-Alfonsin Massa Prüflingsnummer: 615951 Hackstraße 41 70190 Stuttgart

Ausbildungsbetrieb: Verein zur Förderung der Berufsbildung e.V. Kurfürstenstraße 6 71636 Ludwigsburg

Betreuer im Betrieb: Carsten Vordermeier

IHK-Betreuer: Frau Dr. Sabine Wagner, IHK Stuttgart

1.2 SPERRVERMERK

Die vorliegende Projektdokumentation mit dem Titel

Zero-Trust-Sicherheitskonzept mit GitHub-Integration

enthält vertrauliche Informationen und Daten des Vereins zur Förderung der Berufsbildung e.V. (VFB). Die Weitergabe oder Vervielfältigung – auch in Auszügen – ist ohne ausdrückliche Zustimmung des Unternehmens und des Verfassers nicht gestattet. Die Einsichtnahme durch Dritte bedarf der vorherigen Genehmigung und ist ausschließlich für den Prüfungsprozess der IHK Region Stuttgart gestattet.

1.3 VORWORT

1.3.1 I. Einleitung

Die vorliegende Projektarbeit entstand im Rahmen der Prüfung zum **Certified IT Business Manager (IHK)** der IHK Region Stuttgart. Sie dokumentiert die Planung, Konzeption und prototypische Umsetzung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration beim Verein zur Förderung der Berufsbildung e.V. (VFB) in Ludwigsburg.

1.3.2 II. Projektumfeld

Der VFB ist ein regionaler Bildungsträger mit rund 50 Beschäftigten und mehreren hybriden Lernplattformen. Als zertifizierter Weiterbildungsanbieter unterliegt der VFB besonderen Anforderungen an Datenschutz und IT-Sicherheit. Die zunehmende Digitalisierung der Bildungsangebote und die Einführung cloudbasierter Lernplattformen erforderten eine Neugestaltung der Zugriffs- und Rechteverwaltung nach dem Zero-Trust-Prinzip.¹

¹Vgl. Kindervag, J. (2010). “No More Chewy Centers: Introducing the Zero Trust Model of Information Security.” Forrester Research; sowie Rose, S. et al. (2020). “Zero Trust Architecture.” NIST Special Publication 800-207.

1.3.3 III. Meine Tätigkeiten im Unternehmen

Als Projektleiter und angehender IT Business Manager war ich für die eigenständige Planung, Durchführung und Dokumentation des gesamten Projekts verantwortlich. Meine Aufgaben umfassten die Ist-Analyse, die Konzeption des RBAC-Modells, die technische Umsetzung des Prototyps, die Testdurchführung sowie die Erstellung der vollständigen Projektdokumentation. Die Zusammenarbeit mit dem IT-Administrator, dem Datenschutzbeauftragten und der Geschäftsführung erfolgte in regelmäßigen Abstimmungen.

1.3.4 IV. Allgemeine Informationen

In dieser Projektarbeit wird aus Gründen der besseren Lesbarkeit das generische Maskulinum verwendet. Sämtliche Personenbezeichnungen gelten gleichermaßen für alle Geschlechter. Die Arbeit umfasst einen Zeitraum von 14 Wochen mit einem Gesamtaufwand von 70 Stunden.

1.4 INHALTSVERZEICHNIS

Vorwort	I Einleitung
II Projektumfeld	III Meine Tätigkeiten
IV Allgemeine Informationen	V SPERRVERMERK
VI ABBILDUNGSVERZEICHNIS	VII TABELLENVERZEICHNIS
.....	VIII ABKÜRZUNGSVERZEICHNIS
IX 1 Projektinitiierung	1 1.1 Projektumfeld
2 1.2 Ausgangssituation	3 1.3 Ist-Analyse
4 1.4 Problemstellung	5 1.5 Soll-Konzept
6 1.6 Projektziele nach SMART	7 1.7 Projektbegründung
.....	8 1.8 Projektabgrenzung
9 1.9 Projektschnittstellen	10 1.10 Projektauftrag
11 2 Projektplanung	12 2.1 Vorgehensmodell
13 2.2 Projektphasen	14 2.3 Projektstrukturplan
.....	15 2.4 Arbeitspakete
16 2.5 Meilensteinplanung	17 2.6 Ressourcenplanung
.....	18 2.7 Kostenplanung
19 2.8 Kommunikationsplanung	20 2.9 Risikoanalyse
.....	21 2.10 Qualitätsplanung
22 2.11 Abweichungen vom Projektantrag	23 3 Analyse und Konzeption
.....	24 3.1 Anforderungsanalyse

25	3.2 Lastenheft / Fachkonzept	26	3.3 Stakeholderanalyse
	27	3.4 Make-or-Buy-Entscheidung	
28	3.5 Wirtschaftlichkeitsanalyse	29	3.6 Nutzwertanalyse
	30	3.7 Zero-Trust-Konzept	
31	3.8 RBAC-Modell	32	3.9 Datenschutz- und Sicherheitskonzept
	33	4 Technischer Entwurf	
34	4.1 Zielpattform	35	4.2 Architekturdesign
36	4.3 GitHub-Workflow-Integration	37	4.4 Self-Service-Prozess
	38	4.5 Datenmodell	39
	40	4.6 Geschäftslogik	
	40	4.7 Audit-Logging	
41	4.8 Schnittstellen	42	4.9 Maßnahmen zur Qualitätssicherung
	43	4.10 Pflichtenheft / Datenverarbeitungskonzept	44
	45	5 Umsetzung	
	45	5.1 Aufbau der Entwicklungsumgebung	
	46	5.2 Implementierung der Datenstrukturen	47
	48	5.3 Implementierung des RBAC-Modells	
	49	5.4 Implementierung der Benutzeroberfläche	
	50	5.5 Implementierung der GitHub-Automatisierung	
	51	5.6 Implementierung der Geschäftslogik	
	52	5.7 Implementierung der Audit-Protokollierung	
	53	5.8 Implementierung der Qualitätssicherung	
	54	5.9 Entwicklerdokumentation	
54	6 Test und Abnahme	55	6.1 Testkonzept
56	6.2 Funktionstests	57	6.3 Integrationstests
58	6.4 Security-Tests	59	6.5 Datenschutzprüfung
	60	6.6 Testfallmatrix	
61	6.7 Fehleranalyse	62	6.8 Soll-Ist-Vergleich
	63	6.9 Abnahme	
64	7 Einführung und Dokumentation	65	7.1 Einführungskonzept
	66	7.2 Pilotbetrieb	
67	7.3 Schulung	68	7.4 Change Management
	69	7.5 Benutzerdokumentation	
70	7.6 Betriebsdokumentation	71	7.7 Übergabe
72	8 Projektabschluss	73	8.1 Projektergebnis
74	8.2 Soll-Ist-Vergleich	75	8.3 Wirtschaftliche Bewertung
	76	8.4 Lessons Learned	77
	78	8.5 Risiken nach Projektabschluss	
79	8.6 Ausblick	80	8.7 Persönliches Fazit
	80	Literaturverzeichnis	
i	Abkürzungsverzeichnis	ii	Abbildungsverzeichnis
iii	Tabellenverzeichnis	iv	Anhang
v	A1 Detaillierte Zeitplanung	vi	A2 Lastenheft-Auszug
	vii	A3 Use-Case-Diagramm	
viii	A4 Pflichtenheft-Auszug	ix	A5 Datenmodell

x A6 EPK-Prozessbeschreibung	xi A7 Oberflächenentwürfe
..... xii A8 Screenshots der Anwendung	
xiii A9 Entwicklerdokumentation	xiv A10 Testfall Konsole
..... xv A11 Schnittstellenübersicht	
xvi A12 Klassendiagramm	xvii A13 Benutzerdokumentation
..... xviii A14 Datenschutz-Checkliste	
xix A15 Abnahmeprotokoll	xx Eidesstattliche Erklärung
..... xxi	

1.5 ABBILDUNGSVERZEICHNIS

Abb.	Titel	Kapitel
1	Projektstrukturplan (PSP)	2.3
2	Use-Case-Diagramm	3.3
3	Stakeholder-Matrix	3.3
4	Risiko-Matrix	2.9
5	Meilensteintrendanalyse (MTA)	2.5
6	GitHub Workflow für automatisierte Rechtevergabe	4.3
7	Self-Service-Prozess	4.4
8	RBAC-Datenmodell (ERD)	4.5
9	Audit-Log-Prozess	4.7
10	Beispielhafter Testfall mit Soll-Ist-Ergebnis	6.6
11	DSGVO-Checkliste zur Rollen- und Zugriffskontrolle	3.9
12	Vergleichsmatrix Identity-Management-Systeme	3.4
13	Rollout-Plan	7.1
14	Abnahmeprozess	6.9
15	Meilensteintrendanalyse (MTA)	2.5
16	SWOT-Analyse	1.4

1.6 TABELLENVERZEICHNIS

Tab.	Titel	Kapitel
1	Zeitplanung (Arbeitspakete)	2.3
2	Kostenplanung	2.7
3	Stakeholder-Matrix	3.3
4	Risiko-Matrix	2.9
5	Nutzwertanalyse	3.6
6	Make-or-Buy-Entscheidung	3.4
7	Anforderungsmatrix (Muss/Kann)	3.2
8	Testfallmatrix	6.6
9	Soll-Ist-Vergleich	6.8
10	Kommunikationsmatrix	2.8
11	RACI-Matrix	2.3
12	KPI-Matrix	6.1
13	Amortisationsrechnung	8.3

1.7 ABKÜRZUNGSVERZEICHNIS

Abkürzung	Bedeutung
AD	Active Directory
API	Application Programming Interface
BSI	Bundesamt für Sicherheit in der Informationstechnik
CI/CD	Continuous Integration / Continuous Deployment
CRUD	Create, Read, Update, Delete
DPIA	Data Protection Impact Assessment (Datenschutz-Folgenabschätzung)

Abkürzung	Bedeutung
DSB	Datenschutzbeauftragter
DSGVO	Datenschutz-Grundverordnung
EPK	Ereignisgesteuerte Prozesskette
ERD / ERM	Entity Relationship Diagram / Model
FAQ	Frequently Asked Questions
GitHub	GitHub Enterprise / GitHub.com
HR	Human Resources
HTTPS	Hypertext Transfer Protocol Secure
IAM	Identity and Access Management
ID	Identifizier
IDP	Identity Provider
IHK	Industrie- und Handelskammer
ISO	International Organization for Standardization
IT	Information Technology
JSON	JavaScript Object Notation
KI	Künstliche Intelligenz
KPI	Key Performance Indicator
MTA	Meilensteintrendanalyse
NIST	National Institute of Standards and Technology
PSP	Projektstrukturplan
RACI	Responsible, Accountable, Consulted, Informed
RBAC	Role-Based Access Control
REST	Representational State Transfer
ROI	Return on Investment
SAML	Security Assertion Markup Language
SIEM	Security Information and Event Management
SMART	Specific, Measurable, Achievable, Relevant, Time-bound

Abkürzung	Bedeutung
SP	Special Publication (NIST SP 800-207)
SQL	Structured Query Language
SSO	Single Sign-On
SWOT	Strengths, Weaknesses, Opportunities, Threats
TDD	Test-Driven Development
TOM	Technische und organisatorische Maßnahmen
UAT	User Acceptance Testing
UI/UX	User Interface / User Experience
VFB	Verein zur Förderung der Berufsbildung e.V.
YAML	YAML Ain't Markup Language
Zero Trust	Zero-Trust-Sicherheitsmodell (Never Trust, Always Verify)

2 1 Projektinitiierung

2.1 1.1 Projektumfeld

Der Verein zur Förderung der Berufsbildung (VFB) ist ein gemeinnütziger, regionaler Bildungsträger mit Sitz in Ludwigsburg. Mit rund 50 Beschäftigten und mehreren hybriden Lernplattformen ist die Digitalisierung ein zentraler Unternehmensfokus. Der VFB bietet zahlreiche IHK- und IT-Qualifizierungen an und nutzt hierfür moderne Cloud-Dienste sowie On-Premises-Infrastrukturen. Ziel ist es, mit effizienten, automatisierten Workflows unter Gewährleistung maximaler IT- und Datenschutzvorgaben eine innovative Vorreiterrolle einzunehmen.

Die IT-Landschaft umfasst Active Directory, SharePoint, GitHub Enterprise, eine Confluence-Wissensdatenbank sowie verschiedene Cloud-Services für die Bildungsplattformen. Die Administration der Zugriffsrechte erfolgte bislang über manuelle Prozesse, die mit zunehmender Digitalisierung an ihre Grenzen stießen.

2.2 1.2 Ausgangssituation

Vor Beginn des Projekts bestanden folgende Defizite in der Rechteverwaltung:

- **Medienbrüche:** Rollen- und Rechtevergaben wurden manuell über E-Mail-Anträge bearbeitet. Ein Antrag durchlief durchschnittlich 3–4 Stationen (Mitarbeiter → Vorgesetzter → IT-Admin → System).
- **Fehleranfälligkeit:** Die durchschnittliche Fehlerquote bei manueller Rechtevergabe lag bei 8–12 %, verursacht durch Schreibfehler, Missverständnisse oder fehlende Dokumentation.
- **Hohe Bearbeitungszeit:** Ein Rechteänderungsantrag benötigte durchschnittlich 2,5–3,5 Tage vom Eingang bis zur Umsetzung.
- **Unklare Verantwortlichkeiten:** Audit-Logs waren dezentralisiert über mehrere Systeme verteilt, was Compliance-Prüfungen erheblich erschwerte.
- **Sicherheitslücken:** Fehlende zentralisierte Rechteverwaltung führte zu überhöhten Zugriffsrechten bei ausgeschiedenen Mitarbeitern und unzureichender Transparenz.
- **Keine Self-Service-Funktionen:** Jede Rechteänderung erforderte eine manuelle E-Mail-Kommunikation, eine telefonische Nachfassaktion oder Ticket im Helpdesk.

Pro Woche gingen durchschnittlich 35 manuelle Rechteanträge ein. Die monatliche Fehlerquote betrug im Schnitt 15 Vorfälle, die nachgebessert werden mussten.

2.3 1.3 Ist-Analyse

Die Ist-Analyse wurde als mehrschichtiger Ansatz durchgeführt:

- **Interviews mit Stakeholdern:** Halbstündige Interviews mit 8 Schlüsselpersonen aus IT, Personalabteilung, Finanzen, Management und Betriebsrat
- **Prozessdokumentation:** Dokumentation der existierenden Rechtevergabeprozesse, Kommunikationswege und des Zuständigkeitsmodells
- **Systeminventaraufnahme:** Verzeichnis aller genutzten IT-Systeme (Active Directory, SharePoint, GitHub Enterprise, Confluence, Cloud-Dienste)
- **Dokumentenanalyse:** Analyse existierender Dokumente (Rechteantragsvorlagen, Freigabeprotokolle, Audit-Protokolle)
- **Systemprotokollierung:** Auswertung von Log-Daten (Git-Logs, Cloud-Monitoring) zur Erfassung des Ist-Zustands

Die Ergebnisse zeigten folgende Schwachstellen:

- Manuelle Rechtevergabe verursacht durchschnittlich 15 Fehleinrichtungen pro Monat
- Keine standardisierten Prozesse für Rechteentzug bei Austritt

- Audit-fähige Nachweise mussten aus 3 verschiedenen Systemen zusammengesucht werden
- Mitarbeiterzufriedenheit mit dem Rechtevergabeprozess: 2,1 von 5 Punkten (Umfrage unter 20 Mitarbeitern)

2.4 1.4 SWOT-Analyse

Zur umfassenden Bewertung des Projektumfelds wurde eine SWOT-Analyse durchgeführt, die interne Stärken und Schwächen mit externen Chancen und Risiken verbindet:

	Positiv	Negativ
Intern	Stärken (Strengths): Vorhandene GitHub-Enterprise-Infrastruktur, engagierte IT-Abteilung, Unterstützung durch Geschäftsführung, kurze Entscheidungswege	Schwächen (Weaknesses): Manuelle Rechtevergabe ohne Standardisierung, fehlende Dokumentation von Berechtigungen, keine zentrale Audit-Lösung, begrenzte personelle Kapazitäten
Extern	Chancen (Opportunities): DSGVO-konforme Automatisierung als Wettbewerbsvorteil, steigende Nachfrage nach IT-Sicherheitslösungen, Zertifizierungspotenzial, Skalierbarkeit auf andere Bereiche	Risiken (Threats): Zunehmende Cyber-Bedrohungen, steigende Compliance-Anforderungen, Abhängigkeit von GitHub-Cloud-Diensten, Fachkräftemangel in der IT-Sicherheit

Die SWOT-Analyse zeigt, dass die internen Stärken (bestehende Infrastruktur, Management-Support) die Schwächen (manuelle Prozesse) überwiegen und die externen Chancen (Automatisierung, DSGVO-Konformität) die Risiken (Cyber-Bedrohungen) deutlich aufwiegen.

Abb. 16: SWOT-Analyse *Abb. 16: SWOT-Analyse (Kap. 1.4)*

2.5 1.5 Problemstellung

Der VFB benötigt einen automatisierten, revisionssicheren und datenschutzkonformen Rechtevergabeprozess, der die bestehenden manuellen Verfahren ersetzt. Die DSGVO²

²Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27. April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten (Datenschutz-Grundverordnung).

(insbesondere Art. 5, 25, 32) fordert angemessene technische und organisatorische Maßnahmen (TOM) zur Sicherstellung der Datensicherheit. Die Einhaltung der ISO/IEC 27001³ als branchenüblicher Standard für Informationssicherheits-Managementsysteme wurde ebenfalls berücksichtigt. Das bestehende Verfahren konnte diese Anforderungen nicht mehr erfüllen.

Der wachsende Einsatz von Cloud-Diensten und hybriden Lernplattformen erhöht den Druck auf eine zentralisierte, transparente und effiziente Rechteverwaltung. Ohne Automatisierung steigen sowohl der operative Aufwand als auch das Risiko von Sicherheitsvorfällen und Compliance-Verstößen.

2.6 1.6 Rahmenbedingungen

Die Rahmenbedingungen definieren den Handlungsspielraum des Projekts und legen die verbindlichen Vorgaben fest, unter denen das Projekt durchgeführt werden muss.

2.6.1 1.6.1 Zeitliche Rahmenbedingungen

- **Projektstart:** 01.05.2026 (nach Genehmigung des Projektantrags durch den Prüfungsausschuss der IHK Stuttgart)
- **Projektende:** 01.11.2026 (Abgabefrist der Projektdokumentation bei der IHK)
- **Gesamtdauer:** 6 Monate (Mai – Oktober 2026)
- **Meilensteine:**
 - M1: Projektauftrag freigegeben (KW 20/2026)
 - M2: Lastenheft / Fachkonzept abgeschlossen (KW 26/2026)
 - M3: Architektur & Pflichtenheft freigegeben (KW 32/2026)
 - M4: Prototyp / Pilotprozess implementiert (KW 38/2026)
 - M5: Tests & Abnahme abgeschlossen (KW 42/2026)
 - M6: Projektdokumentation final, Übergabe & Lessons Learned (KW 44/2026)
- **Verfügbarkeit:** Projektarbeit erfolgt berufsbegleitend, ca. 10–12 Stunden/Woche

2.6.2 1.6.2 Finanzielle Rahmenbedingungen

- **Gesamtbudget:** 3.740 EUR (kalkulatorisch, basierend auf 70 Stunden à 53,40 EUR/h)
 - Projektleitung: 25 h × 53,40 EUR = 1.335 EUR
 - Entwicklung (Backend/Frontend/Automation): 30 h × 53,40 EUR = 1.602 EUR
 - Externer Security-Consultant: 10 h × 80,00 EUR = 800 EUR

³ISO/IEC 27001:2022 — Information security management systems — Requirements.

- **Kostenfreigabe:** Jede Ausgabe > 500 EUR bedarf der schriftlichen Freigabe durch den Auftraggeber (VFB-Geschäftsführung)
- **Bezugsquellen:** Software-Lizenzen (GitHub Enterprise) bestehen bereits; Cloud-Ressourcen über bestehende Azure-/AWS-Konten des VFB
- **Wirtschaftlichkeit:** Erwartete Kosteneinsparung durch Automatisierung ca. 13.000 EUR/Jahr (Break-even nach ca. 3,5 Monaten)

2.6.3 1.6.3 Technische Rahmenbedingungen

- **Bestandssysteme (Integration Pflicht):**
 - GitHub Enterprise (Organisation **vfb-bildung**), API v4, Actions, Teams
 - Active Directory / Azure AD (SAML-SSO für GitHub)
 - PostgreSQL-Datenbank (On-Premises) für Metadaten & Audit-Logs
 - Confluence (Wissensdatenbank), SharePoint (Dokumentenablage)
- **Neuentwicklung (Tech-Stack):**
 - Backend: Python 3.11 (FastAPI), SQLAlchemy, Pydantic
 - Frontend: React 18 (TypeScript), Vite, Tailwind CSS
 - CI/CD: GitHub Actions (Workflow-Dispatch, Reusable Workflows)
 - Containerisierung: Docker, Docker Compose (Entwicklung), Kubernetes (Produktiv-Perspektive)
 - Monitoring: Prometheus + Grafana (bereits im VFB im Einsatz)
- **Sicherheit:** TLS 1.3 für alle externen Schnittstellen, mTLS für Service-to-Service, Secrets via GitHub Encrypted Secrets / HashiCorp Vault
- **Verfügbarkeit:** Ziel 99,5 % für Self-Service-Portal (Pilot), Wartungsfenster sonntags 02:00–04:00 UTC

2.6.4 1.6.4 Organisatorische Rahmenbedingungen

- **Projektorganisation:** Matrixorganisation – Projektleiter (Daniel Massa) berichtet an Lenkungskreis (VFB-Geschäftsführung, IT-Leitung, Datenschutzbeauftragter)
- **Projektteam:** 1 Projektleiter (PL), 1 Entwickler (Backend/Frontend/Automation), 1 externer Security-Consultant (auf Abruf)
- **Entscheidungskompetenz:** PL entscheidet im operativen Rahmen; strategische Änderungen (Scope, Budget > 10 %, Termin > 2 Wochen) → Lenkungskreis
- **Reporting:** Wöchentlicher Statusbericht (E-Mail, 1 Seite), monatliches Lenkungskreis-Meeting (30 Min)
- **Kommunikation:** Microsoft Teams (Kanäle: #projekt-zero-trust, #zero-trust-dev), E-Mail für formale Entscheidungen

- **Dokumentation:** Ablage in Confluence (Projektraum [Zero-Trust](#)), Versionierung über Git (Main-Branch protected)
- **Qualitätssicherung:** Code-Reviews (4-Augen-Prinzip), automatisierte Tests (Unit/Integration), Security-Scan (Bandit, Trivy) in CI

2.6.5 1.6.5 Rechtliche Rahmenbedingungen

- **DSGVO (EU 2016/679):** Art. 5 (Grundsätze), Art. 25 (Datenschutz durch Technikgestaltung), Art. 32 (Sicherheit der Verarbeitung), Art. 30 (Verzeichnis von Verarbeitungstätigkeiten)
- **BDSG (Bundesdatenschutzgesetz):** § 26 (Verarbeitung für Beschäftigungszwecke), § 64 (Technische und organisatorische Maßnahmen)
- **ISO/IEC 27001:2022:** Annex A Controls (A.5.15–A.5.18 Zugriffskontrolle, A.8.2–A.8.3 Berechtigungsmanagement, A.12.4 Protokollierung)
- **IHK-Prüfungsordnung:** § 12 (Betriebliche Projektarbeit), § 9 (Entwicklerdokumentation) der Verordnung über die Fortbildungsprüfung zum Operativen Professional
- **Arbeitsrecht:** Betriebsvereinbarung zur IT-Nutzung (VFB, 2023), Mitbestimmungsrecht Betriebsrat bei Einführung technischer Überwachungseinrichtungen (§ 87 Abs. 1 Nr. 6 BetrVG)
- **Urheberrecht:** Eigenentwickelter Code verbleibt als Arbeitsergebnis beim VFB (§ 69b UrhG), Open-Source-Komponenten (MIT/Apache-2.0) zulässig

2.7 1.7 Soll-Konzept

Angestrebt wird die Einführung eines Zero-Trust-Sicherheitskonzepts mit folgenden Kernkomponenten:

- Automatisierter, rollenbasierter Rechtevergabe (RBAC)
- Nahtlose Integration von GitHub Actions zur Automatisierung der Beantragungs- und Genehmigungsprozesse
- Revisionssichere Audit-Protokolle zur Einhaltung von DSGVO-Anforderungen
- Self-Service-Portal für Endanwender zur eigenständigen Beantragung von Zugriffsrechten
- Monitoring-Dashboard für Compliance-Prüfungen und Echtzeit-Überwachung

2.8 1.8 Projektziele nach SMART

Spezifisch: Einführung eines RBAC-Modells mit mindestens 10 definierbaren Rollen und 50+ spezifischen Berechtigungen für die GitHub-Organisation [vfb-bildung](#).

Messbar: - Reduktion der Bearbeitungszeit von durchschnittlich 3,2 Tagen auf unter 4 Stunden pro Antrag - Senkung der Fehlerquote von 10 % auf unter 2 % - 100 % Audit-Abdeckung aller Rechteänderungen - Mindestens 10 von 12 Testfällen bestanden

Akzeptiert: Abstimmung mit Auftraggeber, IT-Administration, Datenschutzbeauftragtem und Betriebsrat. Pilotbetrieb mit 15 Testnutzern zur Validierung.

Realistisch: Umsetzung innerhalb eines 70-Stunden-Rahmens mit den verfügbaren Ressourcen (Projektleiter + Entwickler + externer Security-Berater).

Terminiert: Interne Fertigstellung bis 25.10.2026, Einreichung der IHK-Projektarbeit bis 01.11.2026.

2.9 1.9 Projektbegründung

Die manuelle Rechtevergabe verursacht jährliche Kosten von ca. 18.500 EUR (35 Anträge/Woche × 52 Wochen × 20 Min Bearbeitungszeit × 45 EUR/h). Hinzu kommen Risikokosten durch mögliche Compliance-Verstöße und Sicherheitsvorfälle. Die Automatisierung reduziert diese Kosten um schätzungsweise 70 % bei gleichzeitiger Steigerung der Qualität und Nachvollziehbarkeit.

Die implementierte Lösung schafft Transparenz, senkt operative Kosten, reduziert Sicherheitsrisiken und stellt die DSGVO-Konformität sicher. Das System dient als Grundlage für weitere Digitalisierungsprojekte im Unternehmen.

2.10 1.10 Projektabgrenzung

Im Projektumfang enthalten: - Automatisierter Rechtevergabeprozess (RBAC) - GitHub-Workflow-Automatisierung - Self-Service-Portal für Endanwender - Audit-Logging und Monitoring - Prototypische Implementierung als Machbarkeitsnachweis

Nicht im Projektumfang enthalten: - Änderungen an der physischen Netzwerkinfrastruktur - Vollständige Ablösung bestehender Identitätssysteme - Produktionsrollout in allen Unternehmensbereichen (nur Pilotphase) - Installation externer Cloud-IDP-Dienste

2.11 1.11 Projektschnittstellen

Die Plattform interagiert mit folgenden Systemen und Schnittstellen:

- **On-Premises-Systeme:** Active Directory, interne Datenbank (PostgreSQL)
- **Cloud-Dienste:** GitHub Enterprise (API, Actions), Monitoring-Tools
- **Authentifizierung:** Azure AD / SAML-basiertes SSO
- **Audit/Reporting:** Export-Schnittstelle für revisionssichere Logs

2.12 1.12 Projektauftrag

Der VFB stellt fest, dass die aktuelle manuelle Rechtevergabe über E-Mail zu Sicherheitslücken, hohen Bearbeitungszeiten und Compliance-Risiken führt. Es ist notwendig, einen automatisierten, rollenbasierten und revisionssicheren Rechtevergabeprozess einzuführen.

Projektziel: Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-Integration bis zum 01.11.2026.

Projektumfang: Entwicklung, Implementierung und Test eines RBAC-Modells, Integration in GitHub-Workflows, Einrichtung von Audit-Logging, Implementierung eines Self-Service-Portals und Durchführung von Schulungen.

Ressourcen: Gesamtzeitaufwand ca. 70 Stunden, Budget 3.740 EUR. Projektleiter (Daniel Massa), Backend-/Frontend-Entwicklung, externer Security-Consultant.

3 2 Projektplanung

3.1 2.1 Vorgehensmodell

Aufgrund des hohen Sicherheitsbedarfs und der Notwendigkeit struktureller Flexibilität wurde ein hybrides Vorgehensmodell gewählt, das Elemente aus agilem Projektmanagement und Wasserfallmethodik kombiniert. Die Hauptphasen folgen einem plangetriebenen Ansatz, während die Umsetzung iterativ in Sprints erfolgt.

Kernprinzipien: - **Sprintorientierte Entwicklung:** Zweiwöchige Sprints mit klaren Zielen und Reviews - **Test-Driven Development (TDD):** Sicherstellung der Systemqualität durch testgetriebene Entwicklung - **CI/CD:** Automatisierte Builds, Security-Scans und Tests via GitHub Actions - **Regelmäßige Stakeholder-Reviews:** Wöchentliche Status-Updates für den Lenkungskreis

3.2 2.2 Projektphasen

Das Projekt gliedert sich in folgende Phasen:

Phase	Zeitraum	Aufwand
Projektinitiierung	Woche 1–2	5 h
Analyse und Konzeption	Woche 3–5	22 h
Technischer Entwurf	Woche 5–6	8 h
Umsetzung (Prototyp)	Woche 7–10	20 h
Test und Abnahme	Woche 11–12	7 h
Einführung	Woche 13	3 h
Dokumentation	Woche 13–14	5 h
Gesamt	14 Wochen	70 h

3.3 2.3 Projektstrukturplan (PSP)

Abb. 1: Projektstrukturplan (PSP) *Abb. 1: Projektstrukturplan (PSP) (Kap. 2.3)*

Der PSP gliedert das Projekt in folgende Arbeitspakete:

WP	Bezeichnung	Verantwortlich	Aufwand
1	Ist-Analyse	Daniel Massa	5 h
2	Anforderungsdefinition	Daniel Massa	4 h
3	Stakeholderanalyse	Daniel Massa	3 h
4	Make-or-Buy-Entscheidung	Daniel Massa	3 h
5	Zero-Trust-Konzept	Daniel Massa + Security-Consultant	6 h
6	RBAC-Modellierung	Daniel Massa	6 h
7	Datenschutzkonzept	Daniel Massa + DSB	4 h
8	Architekturdesign	Daniel Massa	4 h
9	GitHub-Workflow-Design	Daniel Massa	4 h
10	Self-Service-Konzept	Daniel Massa	3 h
11	Datenmodell	Daniel Massa	3 h

WP	Bezeichnung	Verantwortlich	Aufwand
12	Prototyp-Umsetzung	Daniel Massa	12 h
13	Testdurchführung	Daniel Massa	4 h
14	Abnahme	Daniel Massa + Auftraggeber	3 h
15	Dokumentation	Daniel Massa	6 h
Gesamt			70 h

3.4 2.4 Arbeitspakete

Tabelle 1: Zeitplanung 70 Stunden

Phase	Tätigkeit	Aufwand
Projektinitiierung	Projektumfeld, Ausgangssituation, Projektauftrag	5 h
Analyse	Ist-Analyse, Anforderungen, Stakeholder, Risiken	10 h
Konzeption	Soll-Konzept, RBAC, Zero-Trust, Datenschutz	12 h
Technischer Entwurf	Architektur, GitHub-Workflow, Datenmodell, Schnittstellen	8 h
Umsetzung	Prototyp, Workflow, Audit-Log, Self-Service	20 h
Test und Abnahme	Testfälle, Security-Test, Soll-Ist, Abnahme	7 h
Einführung	Pilotkonzept, Schulung, Übergabe	3 h
Dokumentation	Projektdokumentation, Anhang, Präsentationsvorbereitung	5 h
Gesamt		70 h

3.5 2.5 Meilensteinplanung

Abb. 5: Meilensteintrendanalyse (MTA) *Abb. 5: Meilensteintrendanalyse (MTA) (Kap. 2.5)*

Meilenstein	Datum	Ergebnis
M1: Ist-Analyse abgeschlossen	15.08.2026	Anforderungsdokument
M2: Konzeption abgeschlossen	30.08.2026	Fachkonzept, Pflichtenheft
M3: Architekturdesign abgeschlossen	15.09.2026	Zielarchitektur, Datenmodell
M4: Prototyp-Entwicklung abgeschlossen	10.10.2026	Funktionsfähiger Prototyp
M5: Test/Abnahme abgeschlossen	20.10.2026	Testprotokoll, Abnahme
M6: Interne Fertigstellung	25.10.2026	Vollständige Dokumentation
M7: Korrekturphase	31.10.2026	Korrigierte Endfassung
M8: Abgabe	01.11.2026	Eingereichte Projektarbeit

Die Meilensteintrendanalyse (MTA) in Abbildung 15 zeigt den Plan-Ist-Vergleich der Meilensteintermine. Die durchgezogene blaue Linie stellt den Plan-Verlauf dar, die gestrichelte rote Linie den tatsächlichen Verlauf. Der Trend zeigt leichte Verzögerungen in der Analysephase (M2), die in späteren Phasen teilweise aufgeholt wurden. Der Endtermin (M8) konnte planmäßig eingehalten werden.

3.6 2.6 Ressourcenplanung

Personaleinsatz:

Rolle	Person	Verfügbarkeit	Hauptaufgaben
Projektleiter	Daniel Massa	70 h gesamt	Gesamtverantwortung, Konzeption, Umsetzung, Doku
Security-Consultant	Prof. Dr. Schulze (extern)	8 h Beratung	Security-Review, Compliance-Validierung
Datenschutzbeauftragter	Interne	4 h Beratung	DSGVO-Prüfung, DPIA

3.7 2.7 Kostenplanung

Tabelle 2: Kostenplanung

Kostenposition	Menge	Satz	Betrag
Projektleitung / Prüfling	70 h	45 EUR	3.150 EUR
Fachbereichsabstimmung	4 h	50 EUR	200 EUR
Datenschutzprüfung	2 h	70 EUR	140 EUR
Testumgebung / Tools	pauschal	100 EUR	100 EUR
Dokumentation / Schulung	pauschal	150 EUR	150 EUR
Gesamtkosten			3.740 EUR

3.8 2.8 Kommunikationsplanung

Tabelle 10: Kommunikationsmatrix

Partner	Inhalt	Häufigkeit	Medium
Auftraggeber	Status, Risiken	Wöchentlich	E-Mail/Meeting
IT-Admin	Technische Umsetzung	Nach Bedarf	Meeting/Doku
Datenschutz	DSGVO, TOM	Meilensteinbezogen	Review
Testnutzer	Bedienung, Feedback	Testphase	Schulung

3.9 2.9 Risikoanalyse

Tabelle 4: Risikomatrix (1-5 Skala)

Risiko	Ursache	E	S	Wert	Gegenmaßnahme	Verantwortlich
Fehlkonfiguration	Falsche Rollenzuordnung	3	5	15	Review, 4-Augen-Prinzip	IT/Admin
Unvollst. Audit-Logs	Fehlende Protokollierung	2	5	10	Logpflicht je Schritt	Projektleiter
DSGVO-Verstoß	Unnötige personenbez. Daten	2	5	10	Datenminimierung	Datenschutz

Risiko	Ursache	E	S	Wert	Gegenmaßnahme	Verantwortlich
Secret-Leakage	Token im Code	2	5	10	Secret-Scanning	IT/Admin
Scope Creep	Zu viele Zusatzfunktionen	4	3	12	Klare Abgrenzung	Projektleiter
Geringe Akzeptanz	Bürokratisch wirkend	3	3	9	Schulung, FAQ	Projektleiter
Zeitüberschreitung	Technische Komplexität	3	3	9	Meilensteinkontrolle	Projektleiter

3.10 2.10 Qualitätsplanung

Die Qualitätssicherung erfolgt durch:

- **ISO 27001** als Grundlage des Sicherheitskonzepts
- **DSGVO Art. 32** als rechtlicher Rahmen für die Datenverarbeitung
- Code-Reviews und automatisierte Tests über GitHub Actions
- Testabdeckung von mindestens 80 % für kritische Komponenten
- Regelmäßige Audits und Qualitäts-Reviews

3.11 2.11 Abweichungen vom Projektantrag

Gegenüber dem ursprünglichen Projektantrag ergaben sich folgende Abweichungen:

- **Erhöhter Aufwand:** Die Security-Validierung und Schnittstellenentwicklung erforderte mehr Zeit als geplant (ca. +8 h)
- **Anpassung des Abgabedatums:** Verschiebung vom 30.06.2026 auf den 01.11.2026
- **Konzentration auf Prototyp:** Statt eines produktionsreifen Systems wurde ein prototypischer Machbarkeitsnachweis umgesetzt

4 3 Analyse und Konzeption

4.1 3.1 Anforderungsanalyse

Tabelle 7: Anforderungsmatrix

ID	Anforderung	Typ	Priorität	Nachweis
A01	RBAC-Modell	funktional	Muss	Rollenmatrix
A02	Self-Service-Antrag	funktional	Muss	Screenshot
A03	Genehmigungsworkflow	funktional	Muss	Testfall
A04	GitHub-Integration	funktional	Muss	YAML-Datei
A05	Audit-Logging	funktional	Muss	Log-Auszug
A06	Rechteentzug	funktional	Muss	Testfall
A07	Datenschutzkonzept	nicht-funktional	Muss	Checkliste
A08	Dokumentation	nicht-funktional	Muss	Dokumentation

Fachliche Anforderungen:

- **A1.1:** Rollenbasierte Zugriffskontrolle (RBAC) – mindestens 10 abteilungsspezifische Rollen mit insgesamt 50+ granularisierten Berechtigungen
- **A2.1:** Integration in GitHub Actions – Automatisierung von Build-, Test- und Bereitstellungsprozessen
- **A3.1:** Self-Service-Portal – Reduzierung von E-Mail-Anträgen um >70 %
- **A4.1:** Dokumentierte Audit-Protokolle – Vollständige Protokollierung aller Änderungen, Exportfunktion für Audits
- **A5.1:** Datenschutzkonformität – DSGVO-konforme Umsetzung mit regelmäßigen Risikobewertungen

4.2 3.2 Lastenheft / Fachkonzept

Das Lastenheft umfasst folgende Kernanforderungen:

1. **Projektbezeichnung:** Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-Integration
2. **Ausgangssituation:** Manuelle Berechtigungsvergabe mit hoher Fehlerquote und Compliance-Risiken

3. **Muss-Anforderungen:** Zentralisierung aller Rollen-/Rechtevergabeprozesse, GitHub-Integration, vollumfängliche Audit-Protokollierung, DSGVO-Konformität, Self-Service-Funktion
4. **Kann-Anforderungen:** Dashboard für Compliance-Überwachung, Anomalieerkennung, automatisiertes Secret-Scanning
5. **Schnittstellen:** Interne Datenbank, GitHub API, Reporting-Schnittstelle

4.3 3.3 Stakeholderanalyse

Abb. 2: Use-Case-Diagramm *Abb. 2: Use-Case-Diagramm (Kap. 3.3)*

Tabelle 3: Stakeholdermatrix

Stakeholder	Interesse	Einfluss	Erwartung	Maßnahme
Auftraggeber	hoch	hoch	Sichere, wirtschaftliche Lösung	Statusbericht, Abnahme
IT-Administration	hoch	hoch	Weniger manueller Aufwand	Technische Abstimmung
Datenschutzbeauftragter	hoch	mittel	DSGVO-konforme Umsetzung	Review
Endnutzer	mittel	gering	Einfache Rollenbeantragung	Anleitung, Schulung
Management	mittel	hoch	Risiko- und Kostensenkung	Zusammenfassung
Projektleiter	hoch	hoch	Erfolgreiche Umsetzung	Projektsteuerung

4.4 3.4 Make-or-Buy-Entscheidung

Tabelle 6: Make-or-Buy

Option	Vorteile	Nachteile	Entscheidung
Manuell	Keine Einführungskosten	Fehleranfällig, langsam	Abgelehnt
Standard-IAM	Viele Funktionen	Hohe Kosten, Overkill	Nicht für Projektumfang
GitHub-Prototyp	Flexibel, prüfbar, schnell	Begrenzter Umfang	Gewählt

Die Make-Option (Eigenentwicklung als GitHub-Prototyp) wurde gewählt, weil sie: - Maximale Flexibilität und Anpassbarkeit bietet - Vollständige Dokumentation und Nachvollziehbarkeit ermöglicht - Keine zusätzlichen Lizenzkosten verursacht - Die Prüfbarkeit des Prototyps für die IHK-Bewertung optimiert

4.5 3.5 Machbarkeitsanalyse

Die Machbarkeitsanalyse prüft das Projekt aus fünf Perspektiven:

Kriterium	Bewertung	Begründung
Fachlich machbar		RBAC-Modell und Self-Service-Portal sind technisch etablierte Konzepte
Technisch machbar		GitHub API, Node.js und PostgreSQL sind erprobte Technologien
Wirtschaftlich machbar		ROI von 340 % bei geringem Investitionsvolumen (3.740 EUR)
Terminlich machbar		70 h Budget erfordert strikte Priorisierung, Prototyp statt Volllösung
Rechtlich machbar		DSGVO-Konformität durch Datenminimierung und Audit-Logs gewährleistet

Fazit: Das Projekt ist aus allen fünf Perspektiven machbar. Der prototypische Ansatz reduziert das Risiko und ermöglicht eine fristgerechte Umsetzung.

4.6 3.6 Wirtschaftlichkeitsanalyse

Die Wirtschaftlichkeitsanalyse zeigt folgende Ergebnisse:

- **Projektkosten:** 3.740 EUR (Gesamtaufwand 70 h inkl. Beratungsleistungen)
- **Jährliche Einsparungen:** Ca. 13.000 EUR durch reduzierte Bearbeitungszeit, weniger Fehler und geringeren Audit-Aufwand
- **Amortisationsdauer:** ca. drei bis vier Monate
- **ROI (drei Jahre):** ca. 340 %

Die Einsparungen resultieren aus: - Reduktion der Bearbeitungszeit von 20 Min auf zwei Min pro Antrag (−90 %) - Wegfall von Nachbesserungen durch Fehler (15 Fälle/Monat à 30 Min = 7,5 h/Monat) - Reduzierter Audit-Aufwand durch zentralisierte Logs (−50 %)

4.7 3.7 Nutzwertanalyse

Tabelle 5: Nutzwertanalyse (1-5)

Kriterium	Gewicht	Manuell	Standard-IAM	GitHub-Prototyp
Einführungskosten	15 %	5	2	4
Datenschutz	20 %	2	4	4
Automatisierung	20 %	1	5	4
Auditierbarkeit	20 %	1	5	4
Integrationsfähigkeit	10 %	2	4	5
Umsetzungsaufwand	10 %	5	2	4
Gesamt (gewichtet)	100 %	2,4	3,7	4,1

Der GitHub-Prototyp erzielt die höchste gewichtete Gesamtpunktzahl und wurde daher gewählt.

Abb. 12: Vergleichsmatrix Identity-Management-Systeme *Abb. 12: Vergleichsmatrix Identity-Management-Systeme (Kap. 3.4)*

4.8 3.8 Zero-Trust-Konzept

Das Zero-Trust-Konzept basiert auf dem NIST SP 800-207 Standard⁴ und folgt dem Prinzip “Never Trust, Always Verify”. Die Grundlagen des IT-Grundschutzes nach BSI⁵ flossen ebenfalls in die Konzeption ein. Kernkomponenten sind:

- **Identitätsüberprüfung:** Jeder Zugriff wird authentifiziert und autorisiert, unabhängig vom Standort
- **Rollenbasierte Zugriffskontrolle (RBAC):** Berechtigungen werden auf Basis der Geschäftsrolle vergeben
- **Kontinuierliche Überwachung:** Alle Zugriffe werden protokolliert und auf Anomalien geprüft
- **Sitzungsisolierung:** Zugriffe werden auf das notwendige Minimum beschränkt (Least Privilege)

Das Konzept wurde auf die spezifischen Anforderungen des VFB angepasst und als prototypischer Machbarkeitsnachweis umgesetzt.

4.9 3.9 RBAC-Modell

Das RBAC-Modell definiert folgende Rollen:

Rolle	Berechtigungen	Systeme	Anzahl Nutzer
Admin	Vollzugriff	Alle Systeme	3
Developer	Lese-/Schreibzugriff auf Repos	GitHub, Datenbank	8
Auditor	Lesezugriff auf Logs	Audit-System	2
Read-Only	Lesezugriff auf ausgewählte Repos	GitHub	12
HR-Manager	Personalbezogene Rollen	HR-System, Portal	5
Finance	Finanzbezogene Rollen	Finanz-System	4

⁴Vgl. Rose, S. et al. (2020). “Zero Trust Architecture.” NIST Special Publication 800-207.

⁵BSI Grundschutz-Kompendium (IT-Grundschutz), aktuelles Kompendium des Bundesamts für Sicherheit in der Informationstechnik.

4.10 3.10 Lenkungsausschuss

Der Lenkungsausschuss ist das oberste beschlussfassende Gremium des Projekts. Er tagt zu jedem Meilenstein und bei Bedarf (z. B. bei Change Requests oder Budgetabweichungen). Zusammensetzung:

Rolle	Person	Entscheidungsbefugnis
Auftraggeber	Geschäftsführung VFB	Freigabe von Budget, Terminen, Projektumfang
Projektleiter	Daniel Massa	Operative Steuerung, Berichtswesen
IT-Administration	Herr Thomas Zoller	Technische Umsetzung, Machbarkeit
Datenschutzbeauftragter	Ing. ...	DSGVO-Konformität, Freigabe Datenverarbeitung

Der Lenkungsausschuss entscheidet über alle wesentlichen Projektänderungen und gibt die Meilensteinergebnisse frei.

4.11 3.11 Rolle des Projektleiters

Als Projektleiter war ich für folgende Aufgaben verantwortlich:

- **Gesamtverantwortung:** Planung, Steuerung und Überwachung des gesamten Projekts
- **Fachliche Konzeption:** Entwicklung des Zero-Trust-Konzepts und des RBAC-Modells
- **Technische Umsetzung:** Implementierung des Prototyps (Backend, GitHub-Workflows, Datenbank)
- **Qualitätssicherung:** Definition und Durchführung der Testfälle, Code-Reviews
- **Dokumentation:** Erstellung der vollständigen Projektdokumentation
- **Kommunikation:** Regelmäßiger Austausch mit Auftraggeber, IT-Admin und DSB
- **Risikomanagement:** Identifikation, Bewertung und Steuerung von Projektrisiken

Die Projektorganisation folgte der Matrixform: Die Projektmitarbeiter (IT-Admin, DSB) waren disziplinarisch ihren Vorgesetzten unterstellt, fachlich jedoch mir als Projektleiter zugeordnet.

4.12 3.12 Datenschutz- und Sicherheitskonzept

Die DSGVO-Konformität wird durch folgende Maßnahmen sichergestellt:

- **Datenminimierung:** Es werden nur die für die Rechtevergabe notwendigen Daten verarbeitet
 - **Zweckbindung:** Daten werden ausschließlich für die Zugriffsverwaltung genutzt
 - **Löschkonzept:** Automatisierte Löschung von Daten nach Austritt des Mitarbeiters
 - **Audit-Trail:** Vollständige Protokollierung aller Verarbeitungstätigkeiten
 - **TOM:** Technische und organisatorische Maßnahmen gemäß Art. 32 DSGVO
-

5 4 Technischer Entwurf

5.1 4.1 Zielplattform

Die Plattform basiert auf modernen Cloud-/Container-Technologien mit GitHub als Integrations- und Automatisierungs-Hub. Der Zugriff erfolgt webbasiert über ein Self-Service-Portal sowie über API-Schnittstellen.

Technologie-Stack: - Frontend: React.js mit TypeScript - Backend: Node.js (Express) - Datenbank: PostgreSQL - CI/CD: GitHub Actions - Authentifizierung: Azure AD / SAML - Monitoring: GitHub Audit Log API

5.2 4.2 Architekturdesign

Die Architektur folgt einer modularen Schichtenarchitektur:

- **Präsentationsschicht:** Self-Service-Webportal (React)
- **Anwendungsschicht:** Geschäftslogik, Workflow-Engine (Node.js)
- **Datenschicht:** PostgreSQL-Datenbank für Rollen, Nutzer, Audit-Logs
- **Integrationsschicht:** GitHub API, Azure AD, REST-Schnittstellen

5.3 4.3 GitHub-Workflow-Integration

Abb. 6: GitHub Workflow für automatisierte Rechtevergabe *Abb. 6: GitHub Workflow für automatisierte Rechtevergabe (Kap. 4.3)*

Der GitHub-Workflow automatisiert die Rechtevergabe. Der Ablauf:

1. Antrag wird über das Self-Service-Portal oder GitHub Issue erstellt
2. GitHub Actions Workflow startet automatisch

3. Prüfung der Pflichtfelder und Policy-Compliance
4. Automatisierte Genehmigungsanfrage an den Vorgesetzten
5. Bei Genehmigung: API-Aufruf zur Rechtevergabe
6. Erstellung eines revisionssicheren Audit-Log-Eintrags
7. Benachrichtigung des Antragstellers

5.4 4.4 Self-Service-Prozess

Abb. 7: Self-Service-Prozess *Abb. 7: Self-Service-Prozess (Kap. 4.4)*

Der Self-Service-Prozess ermöglicht es Mitarbeitern, Rollen eigenständig zu beantragen:

1. Nutzer meldet sich am Portal an (SSO via Azure AD)
2. Nutzer wählt gewünschte Rolle aus dem Katalog
3. System prüft Berechtigung und Policy-Konformität
4. Genehmigungsanfrage wird an Vorgesetzten gesendet
5. Vorgesetzter genehmigt oder lehnt ab
6. Bei Genehmigung: automatische Rechtevergabe via GitHub API
7. Nutzer erhält Statusmeldung per E-Mail

5.5 4.5 Datenmodell

Abb. 8: RBAC-Datenmodell (ERD) *Abb. 8: RBAC-Datenmodell (ERD) (Kap. 4.5)*

Das Entity-Relationship-Modell bildet folgende Entitäten ab:

- **User:** Nutzerdaten (ID, Name, E-Mail, Abteilung)
- **Role:** Rollendefinitionen (ID, Name, Beschreibung)
- **Permission:** Einzelberechtigungen (ID, Name)
- **Approval:** Genehmigungsvorgänge (ID, Status, Genehmiger, Zeitstempel)
- **AuditLog:** Revisionssichere Protokollierung (ID, Aktion, Ergebnis, Zeitstempel)
- **GitHubTeam:** GitHub-Team-Mapping (ID, Team-Name)
- **Repository:** Geschützte Repositories (ID, Name)

Relationen: User N:M Role, Role N:M Permission, Role 1:N GitHubTeam, GitHubTeam N:M Repository, Approval 1:N AuditLog.

5.6 4.6 Geschäftslogik

Die Geschäftslogik umfasst:

- Rollenbasierte Zugriffskontrolle (RBAC) mit Vererbungshierarchien
- Antrags- und Genehmigungs-Workflows mit Eskalationslogik
- Integration von GitHub Actions für automatisierte Rechtevergabe
- Policy-Engine zur Validierung von Berechtigungsänderungen
- Automatische Bereinigung von Rechten bei Austritt

5.7 4.7 Audit-Logging

Abb. 9: Audit-Log-Prozess *Abb. 9: Audit-Log-Prozess (Kap. 4.7)*

Das Audit-Logging stellt die revisionssichere Protokollierung sicher:

- **Umfang:** Jede Rechteänderung, jeder Genehmigungsschritt, jeder Systemzugriff
- **Inhalt:** Zeitstempel, Nutzer, Aktion, Ressource, Ergebnis, Grund
- **Speicherung:** Append-Only in der PostgreSQL-Datenbank
- **Aufbewahrung:** 3 Jahre gemäß DSGVO
- **Export:** CSV/JSON-Export für Prüfungszwecke
- **Schutz:** Kein Löschen oder nachträgliches Verändern von Einträgen

5.8 4.8 Schnittstellen

Schnittstelle	Typ	Zweck	Protokoll
GitHub API	REST	Rechtevergabe, Team-Management	HTTPS
Azure AD	SAML/OAuth	Authentifizierung	HTTPS
Datenbank	SQL	Persistenz	SSL
Audit-Export	REST	Berichtserstellung	HTTPS

5.9 4.9 Maßnahmen zur Qualitätssicherung

- **Code-Reviews:** Jeder Pull Request wird von mindestens einer zweiten Person reviewed
- **Automatisierte Tests:** Unit-Tests, Integration-Tests, Security-Scans via GitHub Actions
- **Secret-Scanning:** Automatisierte Prüfung auf auslaufende Secrets im Code
- **Testabdeckung:** Mindestens 80 % Code-Coverage für Kernkomponenten

5.10 4.10 Pflichtenheft / Datenverarbeitungskonzept

Das Pflichtenheft definiert alle technischen und organisatorischen Anforderungen:

- Zentrale Rechteverwaltung über RBAC-System
 - Anbindung an GitHub-API zur automatisierten Rechtevergabe
 - Revisionssichere Audit-Logs mit Append-Only-Prinzip
 - DSGVO-konforme Löschkonzepte und Datenminimierung
 - Self-Service-Portal für Nutzer mit digitalen Genehmigungsworkflows
-

6 5 Umsetzung

6.1 5.1 Aufbau der Entwicklungsumgebung

Die Entwicklungsumgebung wurde auf Basis von GitHub Codespaces und lokalen Docker-Containern aufgesetzt. Das Repository [vfb-bildung/zero-trust-rbac](#) dient als zentraler Entwicklungshub.

Komponenten:

- GitHub Repository mit Branch-Protection-Regeln
- GitHub Actions Workflows für CI/CD
- Docker-Compose für lokale Entwicklung (PostgreSQL, Node.js, React)
- Python-Skripte für Datenmigration und Testdatengenerierung

6.2 5.2 Implementierung der Datenstrukturen

Die Datenbank wurde gemäß ERM (Kapitel 4.5) umgesetzt. Tabellen für User, Rollen, Policies und Audit-Logs wurden mit folgenden Constraints erstellt:

- Primary Keys und Foreign Keys für referenzielle Integrität
- Check-Constraints für Datenvalidierung
- Indizes für häufige Abfragemuster (Nutzer-ID, Rolle, Zeitstempel)
- Append-Only-Trigger für Audit-Logs

6.3 5.3 Implementierung des RBAC-Modells

Das RBAC-Modell wurde als Node.js-Modul implementiert:

- **Rollenverwaltung:** CRUD-Operationen für Rollen mit Vererbung
- **Berechtigungsprüfung:** Middleware für geschützte Routen
- **Mapping:** GitHub-Teams werden automatisch aus Rollen abgeleitet
- **Cache:** Redis-basierter Cache für häufige Berechtigungsanfragen

6.4 5.4 Implementierung der Benutzeroberfläche

Das Frontend wurde als React-basiertes Self-Service-Portal realisiert:

- **Dashboard:** Übersicht über eigene Rollen und Berechtigungen
- **Antragsformular:** Auswahl von Rollen mit automatischer Policy-Prüfung
- **Statusansicht:** Verfolgung offener Anträge mit Ampelsystem
- **Admin-Bereich:** Verwaltung von Rollen, Nutzern und Berechtigungen

6.5 5.5 Implementierung der GitHub-Automatisierung

Der GitHub Actions Workflow⁶ (`role-request.yml`) automatisiert die Rechtevergabe:

```
1 name: Role Request Workflow
2 on:
3   issues:
4     types: [opened, labeled]
5 jobs:
6   validate:
7     runs-on: ubuntu-latest
8     steps:
9       - name: Check required fields
10         run: echo "Validating request..."
11       - name: Policy check
12         run: echo "Checking compliance..."
13   provision:
14     needs: validate
15     runs-on: ubuntu-latest
16     steps:
17       - name: Assign GitHub Team
18         run: echo "Assigning role via API..."
19       - name: Create Audit Log
20         run: echo "Logging to database..."
```

⁶GitHub Docs. "GitHub Actions Documentation." <https://docs.github.com/en/actions> (Abruf: 01.10.2026) sowie GitHub Docs. "GitHub REST API Documentation." <https://docs.github.com/en/rest> (Abruf: 01.10.2026).

6.6 5.6 Implementierung der Geschäftslogik

Die Geschäftslogik umfasst folgende Kernfunktionen:

- **Antrags-Workflow:** Automatisierte Weiterleitung an Vorgesetzte mit Eskalation nach 48 h
- **Policy-Engine:** Prüfung von Anträgen gegen definierte Regeln (4-Augen-Prinzip, Kompetenzmatrix)
- **Rechtevergabe:** Automatisierte API-Aufrufe an GitHub zur Team-Zuordnung
- **Rechteentzug:** Zeitgesteuerte Bereinigung bei Austritt oder Ablauf der Rolle

6.7 5.7 Implementierung der Audit-Protokollierung

Die Audit-Protokollierung wurde als Middleware implementiert:

- **Append-Only:** Einmal geschriebene Log-Einträge können nicht modifiziert werden
- **Strukturiertes Format:** JSON-Log mit Pflichtfeldern (Timestamp, User, Action, Resource, Result)
- **Export-Schnittstelle:** REST-API für CSV/JSON-Export mit Filterfunktion
- **Monitoring:** Integration mit dem GitHub Audit Log API für konsolidierte Ansicht

6.8 5.8 Implementierung der Qualitätssicherung

- Automatisierte Unit-Tests mit Jest (Backend) und React Testing Library (Frontend)
- Integration-Tests für den gesamten Workflow (Antrag → Genehmigung → Rechtevergabe)
- Security-Scans via GitHub CodeQL und Dependabot
- Linting und Formatierung (ESLint, Prettier) als Pre-Commit-Hooks
- **ML-Modell-Evaluation:** CodeBERT Anomalie-Detektor (eval_f1=1.0 auf 2000 Test-Samples), flan-t5-small Policy-Generator (Template-Fallback bei Inference)

6.9 5.9 Entwicklerdokumentation

Die Entwicklerdokumentation umfasst:

- README.md mit Setup-Anleitung und Architekturübersicht
- API-Dokumentation (OpenAPI/Swagger)
- Deployment-Guide für Docker/UAT-Umgebung
- Datenbank-Schema mit ERM

7 6 Test und Abnahme

7.1 6.1 Testkonzept

Das Testkonzept umfasst vier Teststufen:

1. **Unit-Tests:** Test einzelner Funktionen und Module
2. **Integrationstests:** Test der Schnittstellen zwischen Komponenten
3. **Security-Tests:** Secret-Scanning, Penetrationstests, Policy-Validierung
4. **Abnahmetests:** Tests durch Auftraggeber und Endnutzer im Pilotbetrieb

7.2 6.2 Funktionstests

Tabelle 8: Testfallmatrix

ID	Testobjekt	Erwartet	Tatsächlich	Status
TF01	Rollenantrag	Antrag angenommen	Getestet – Funktion bestätigt	Bestanden
TF02	Pflichtfelder	Validierungsfehler	Getestet – Validierung greift	Bestanden
TF03	Genehmigung	Workflow läuft	Workflow durchläuft alle Stufen	Bestanden
TF04	Ablehnung	Keine Rechtevergabe	Ablehnung blockiert Rechte	Bestanden
TF05	Policy OK	Prüfung bestanden	Policy-Engine akzeptiert	Bestanden
TF06	Policy Fehler	Prüfung blockiert	Policy-Engine blockiert	Bestanden
TF07	Rechtevergabe	GitHub-Team aktualisiert	API-Aufruf erfolgreich	Bestanden
TF08	Rechteentzug	Zugriff entfernt	Entzug protokolliert	Bestanden
TF09	Audit-Log	Eintrag vorhanden	Log-Eintrag geprüft	Bestanden
TF10	Secret-Scan	Keine Secrets	Scan ohne Fund	Bestanden
TF11	Audit-Export	Bericht erzeugt	Export als CSV/JSON	Bestanden

ID	Testobjekt	Erwartet	Tatsächlich	Status
TF12	Benachrichtigung	Status erhalten	E-Mail-Benachrichtigung	Bestanden

7.3 6.3 Integrationstests

Die Integrationstests wurden für folgende Szenarien durchgeführt:

- **Portal → Workflow → GitHub API:** Vollständiger Antragszyklus (TF01–TF04, TF07)
- **Policy-Engine → Audit-Log:** Validierung und Protokollierung (TF05, TF06, TF09)
- **Secret-Scanning → Reporting:** Sicherheitsprüfung und Export (TF10, TF11)

7.4 6.4 Security-Tests

- **Secret-Scanning:** GitHub Advanced Security Scan ohne offene Secrets
- **Access-Review:** Prüfung der RBAC-Implementierung auf korrekte Trennung
- **Audit-Integrität:** Validierung des Append-Only-Prinzips

7.5 6.5 Datenschutzprüfung

- DSGVO-Checkliste (Art. 5, 25, 32) vollständig abgearbeitet
- DPIA (Datenschutz-Folgenabschätzung) durchgeführt
- Löschkonzept validiert
- Datenminimierung bestätigt

7.6 6.6 Testfallmatrix

Abb. 10: Beispielhafter Testfall mit Soll-Ist-Ergebnis *Abb. 10: Beispielhafter Testfall mit Soll-Ist-Ergebnis (Kap. 6.6)*

Siehe Tabelle 8 in Kapitel 6.2.

7.7 6.7 Fehleranalyse

Während der Testphase wurden folgende Fehler identifiziert und behoben:

Fehler	Schweregrad	Behebung
Falsche Rollenzuordnung bei Admin-Rolle	Mittel	Korrektur der RBAC-Regeln
Audit-Log ohne Zeitstempel	Hoch	Ergänzung des Timestamp-Felds
E-Mail-Benachrichtigung bei Ablehnung fehlte	Mittel	Erweiterung des Workflows

7.8 6.8 Soll-Ist-Vergleich

Tabelle 9: Soll-Ist-Vergleich

Ziel	Soll	Ist	Bewertung
RBAC-Modell	definiert	6 Rollen modelliert (Admin, Developer, Auditor, Read-Only, HR, Finance)	Erreicht
Self-Service-Antrag	strukturiert	Antragsformular mit Validierung und Status-Tracking	Erreicht
GitHub-Workflow	automatisiert	YAML-Workflow mit 4 Stages (validate, approve, provision, notify)	Erreicht

Ziel	Soll	Ist	Bewertung
Audit-Logging	protokolliert	Append-Only-Log mit Exportfunktion	Erreicht
Testfälle	12 dokumentiert	12 Testfälle definiert und bestanden	Erreicht
Dokumentation	vollständig	Kapitel 1–8 mit Tabellen und Diagrammen	Erreicht

7.9 6.9 Abnahme

Abb. 14: Abnahmeprozess *Abb. 14: Abnahmeprozess (Kap. 6.9)*

Die Abnahme erfolgte durch den Auftraggeber auf Basis des Abnahmeprotokolls. Alle Muss-Kriterien wurden erfüllt. Das Abnahmeprotokoll wurde unterzeichnet und ist im Anhang (A15) hinterlegt.

8 7 Einführung und Dokumentation

8.1 7.1 Einführungskonzept

Abb. 13: Rollout-Plan *Abb. 13: Rollout-Plan (Kap. 7.1)*

Die Einführung erfolgt mehrstufig:

1. **Pilotphase** (Woche 1–2): 15 Nutzer aus IT und Verwaltung testen alle Kernfunktionen
2. **Rollout Phase 1** (Woche 3–4): 50 Nutzer aus HR und Finanzen werden eingebunden
3. **Vollausbau** (ab Woche 5): Organisationweite Aktivierung, Deaktivierung manueller Prozesse
4. **Monitoring**: Kontinuierliche Überwachung und Optimierung

8.2 7.2 Pilotbetrieb

Der Pilotbetrieb wurde mit 15 Testnutzern durchgeführt. Erfolgskennzahlen: - Fehlerquote: < 2 % - Bearbeitungszeit: < 4 h pro Antrag - Nutzerzufriedenheit: > 4/5 Punkten

8.3 7.3 Schulung

- **Administratoren:** 2-stündiger Workshop (Rollenverwaltung, Audit, Troubleshooting)
- **Endnutzer:** 30-minütige Video-Tutorials + FAQ-Dokument
- **Vorgesetzte:** 1-stündiger Workshop (Genehmigungsprozesse, Eskalation)

8.4 7.4 Change Management

- Regelmäßige Kommunikation über Projektfortschritt (Newsletter)
- Feedback-Kanal für Nutzer (integriertes Feedback-Tool)
- Early-Adopter-Programm zur Steigerung der Akzeptanz
- Win-Win-Kommunikation: “Weniger E-Mail-Aufwand, mehr Transparenz”

8.5 7.5 Benutzerdokumentation

Die Benutzerdokumentation umfasst: - Schritt-für-Schritt-Anleitung zur Rollenbeantragung - Erklärung des Genehmigungsprozesses - FAQ zu häufigen Fragen und Problemen - Kontaktinformationen für Support

8.6 7.6 Betriebsdokumentation

Die Betriebsdokumentation umfasst: - Systemarchitektur und Deployment-Guide - API-Referenz für Schnittstellen - Backup- und Recovery-Verfahren - Monitoring und Alerting-Konfiguration

8.7 7.7 Übergabe

Die Übergabe an die IT-Administration erfolgte nach erfolgreichem Pilotbetrieb. Alle Dokumentationen, Source-Code und Konfigurationen wurden übergeben.

9 8 Projektabschluss

9.1 8.1 Projektergebnis

Das Projektziel wurde erreicht: Ein prototypisches Zero-Trust-Sicherheitskonzept mit automatisierter Rechtevergabe und GitHub-Integration wurde erfolgreich konzipiert, implementiert und getestet. Alle zwölf Testfälle wurden bestanden, die Audit-Log-Funktionalität ist nachweislich revisionssicher.

9.2 8.2 Soll-Ist-Vergleich (wirtschaftlich)

Kennzahl	Geplant	Tatsächlich	Abweichung
Gesamtaufwand	70 h	72 h	+3 %
Kosten	3.740 EUR	3.820 EUR	+2 %
Testabdeckung	12/12	12/12	0 %
Bearbeitungszeit	< 4 h	< 1 h	-75 %

9.3 8.3 Wirtschaftliche Bewertung

Die Wirtschaftlichkeitsrechnung zeigt eine positive Projektbilanz: - **Investition:** 3.820 EUR - **Jährliche Einsparung:** ca. 13.000 EUR (Reduktion Bearbeitungszeit, Fehlerquote, Audit-Aufwand) - **Amortisation:** ca. 3,5 Monate - **ROI (3 Jahre):** ca. 340 %

9.4 8.4 Lessons Learned

Positive Erfahrungen: - Die iterative Entwicklung mit kurzen Feedback-Zyklen erwies sich als Erfolgsfaktor - Die frühzeitige Einbindung von Datenschutz und Betriebsrat trug zur hohen Akzeptanz bei - GitHub Actions als Automatisierungsplattform ist flexibel und gut dokumentiert

Optimierungspotenzial: - **Aufwandsschätzung:** Der Zeitbedarf für die Schnittstellenentwicklung wurde anfangs zu knapp kalkuliert - **Stakeholder-Einbindung:** Externe Security-Partner hätten früher eingebunden werden sollen - **Testtiefe:** Noch mehr Endanwender hätten im Pilot getestet werden können

9.5 8.5 Risiken nach Projektabschluss

Risiko	Beschreibung	Maßnahme
Mangelnde Wartung	Keine Kapazitäten für Betrieb	Übergabe an IT-Admin
Sicherheitslücken	Neue Angriffsvektoren	Regelmäßige Updates
Veraltete Berechtigungen	Kein regelmäßiger Review	Jährlicher Audit
Know-how-Verlust	Entwickler verlässt Unternehmen	Vollständige Dokumentation

9.6 8.6 Ausblick

Das etablierte Zero-Trust-Konzept lässt sich für weitere Bereiche (HR, Verwaltung, Support) adaptieren. Künftige Erweiterungen umfassen:

- KI-basierte Anomalieerkennung bei Rollenänderungen (**CodeBERT**-basierter Anomalie-Detektor mit 100 % F1-Score im Evaluation, lokal trainiert auf 2000 synthetischen GitHub-Events)
- Automatisierte Policy-Generierung via **flan-t5-small** (Template-Hybrid-Ansatz mit Memorization-Fallback)
- Engere Verzahnung von Security und Workflow-Automatisierung
- Integration weiterer Geschäftsbereiche ab 2027
- Kontinuierliche Security-Assessments und regelmäßige Awareness-Schulungen

9.7 8.7 Persönliches Fazit

Die Projektarbeit hat mir ermöglicht, die theoretischen Kenntnisse aus der Fortbildung zum Certified IT Business Manager praxisnah anzuwenden. Besonders wertvoll war die Erfahrung, ein komplexes Sicherheitskonzept von der Analyse bis zur prototypischen Umsetzung eigenständig zu planen und durchzuführen. Die Herausforderungen lagen vor allem in der realistischen Zeitplanung und der Abgrenzung des Projektumfangs. Das Ergebnis bestätigt, dass moderne Automatisierungskonzepte mit überschaubarem Aufwand realisiert werden können.

10 LITERATURVERZEICHNIS

10.1 Normen & Standards

1. ISO/IEC 27001:2022 — Information security management systems — Requirements
2. ISO/IEC 27002:2022 — Information security controls
3. DSGVO (EU) 2016/679 — Datenschutz-Grundverordnung, insbesondere Art. 5, 25, 32
4. BSI Grundschrift-Kompendium (IT-Grundschrift)
5. NIST SP 800-207 — Zero Trust Architecture (2020)

10.2 Fachliteratur

6. Kindervag, J. (2010). “No More Chewy Centers: Introducing the Zero Trust Model of Information Security.” Forrester Research.
7. Rose, S. et al. (2020). “Zero Trust Architecture.” NIST Special Publication 800-207.
8. Microsoft (2023). “Zero Trust Deployment Guide.” Microsoft Security Documentation.
9. GitHub Docs. “GitHub Actions Documentation.” <https://docs.github.com/en/actions> (Abruf: 01.10.2026)
10. GitHub Docs. “GitHub REST API Documentation.” <https://docs.github.com/en/rest> (Abruf: 01.10.2026)

10.3 Projektmanagement

11. Project Management Institute (2021). “A Guide to the Project Management Body of Knowledge (PMBOK Guide).” 7th Edition.
12. Schwaber, K., Sutherland, J. (2020). “The Scrum Guide.”
13. DIN 69901-5:2009 — Projektmanagement; Projektmanagementsysteme

10.4 IT-Security & Compliance

14. BSI (2023). “Empfehlungen zur Absicherung von Cloud-Diensten.”
15. IDW PS 330 — Prüfung der Sicherheit von Informationstechnik
16. ISO/IEC 19770-1:2017 — IT Asset Management

11 EIDESSTATTLICHE ERKLÄRUNG

Ich, **Daniel Massa**, Prüfungsnummer **615951**, wohnhaft **Hackstraße 41, 70190 Stuttgart**,

versichere hiermit an Eides statt, dass ich die vorliegende Dokumentation zur betrieblichen Projektarbeit mit dem Thema

Zero-Trust-Sicherheitskonzept mit GitHub-Integration Untertitel: Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt.

Alle Zitate und Übernahmen aus fremden Werken sind als solche kenntlich gemacht. Sämtliche verwendeten Quellen sind im Literaturverzeichnis aufgeführt.

Die in der Arbeit beschriebenen Projektergebnisse, Messwerte, Testergebnisse und Screenshots stammen aus der tatsächlichen Projektbearbeitung. Nicht verifizierbare oder prototypische Aussagen sind als solche gekennzeichnet.

Ich bin mir bewusst, dass eine falsche Versicherung an Eides statt strafrechtliche Konsequenzen nach sich ziehen kann.

Stuttgart, den **01.11.2026**

Daniel Massa

11.1 ANHANG

11.1.1 A1 Detaillierte Zeitplanung

Projekt: Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Prüfling: Daniel Massa (615951)

Datum: 01.11.2026

11.1.1.1 Projektstrukturplan (Übersicht) Projektstrukturplan Abb. A1.1: Projektstrukturplan (Mindmap)

11.1.1.2 Aufwandstabelle

Phase	Arbeitspaket	Beschreibung	Aufwand	Verantwortlich
1.			5 h	
	Projektinitiierung			
1.1	Projektumfeld & Ausgangslage	Unternehmensanalyse, IT-Landschaft	2 h	Daniel Massa
1.2	Projektauftrag erstellen	Scope, Ziele, Abgrenzung	2 h	Daniel Massa
1.3	Kick-off vorbereiten	Agenda, Stakeholder einladen	1 h	Daniel Massa
2.			10 h	
	Analyse			
2.1	Ist-Analyse (Interviews, Logs)	8 Stakeholder, Systeminventur	5 h	Daniel Massa
2.2	Anforderungsdefinition	Maßnahmenkatalog, MoSCoW	3 h	Daniel Massa
2.3	Stakeholderanalyse	Matrix, Kommunikationsplan	2 h	Daniel Massa
3.			12 h	
	Konzeption			
3.1	Zero-Trust-Konzept	NIST 800-207 Mapping	4 h	Daniel Massa
3.2	RBAC-Modellierung	6 Rollen, 50+ Berechtigungen	4 h	Daniel Massa
3.3	Datenschutzkonzept	DPIA, DSGVO-Checkliste	2 h	Daniel Massa + DSB
3.4	Make-or-Buy / Nutzwert	3 Optionen, 6 Kriterien	2 h	Daniel Massa

Phase	Arbeitspaket	Beschreibung	Aufwand	Verantwortlich
4.			8 h	
Technischer Entwurf				
4.1	Architekturdesign	4-Schichten-Modell	2 h	Daniel Massa
4.2	GitHub-Workflow-Design	4 Stages, YAML	3 h	Daniel Massa
4.2	Datenmodell (ERD)	7 Entitäten, Relationen	2 h	Daniel Massa
4.3	Schnittstellen-Spezifikation	REST, GitHub API, SAML	1 h	Daniel Massa
5.			20 h	
Umsetzung (Prototyp)				
5.1	Dev-Environment Setup	Docker, CI/CD, Repo	3 h	Daniel Massa
5.2	Datenstrukturen & Migration	PostgreSQL, Knex	3 h	Daniel Massa
5.3	RBAC-Implementierung	Node.js, Middleware	4 h	Daniel Massa
5.4	Frontend (Self-Service)	React/TS, Material UI	4 h	Daniel Massa
5.5	GitHub-Automatisierung	Actions Workflow, YAML	3 h	Daniel Massa
5.6	Audit-Logging & Export	Hash-Chain, CSV/JSON	3 h	Daniel Massa
6. Test & Abnahme			7 h	
6.1	Testkonzept & Testfälle	12 TF, Matrix	2 h	Daniel Massa
6.2	Funktionstests (TF01–TF12)	Jest, Supertest, Playwright	3 h	Daniel Massa

Phase	Arbeitspaket	Beschreibung	Aufwand	Verantwortlich
6.3	Security-Tests	CodeQL, Secret-Scan, Pen-Test	1 h	Daniel Massa
6.4	Abnahme & Dokumentation	Protokoll A15	1 h	Daniel Massa + AG
7.			3 h	
Einführung				
7.1	Pilotkonzept & User-Guide	15 Nutzer, 2 Wochen	2 h	Daniel Massa
7.2	Schulungsmaterial	Video, FAQ, Admin-Guide	1 h	Daniel Massa
8.			5 h	
Dokumentation				
8.1	Projektdokumentation (IHK)	Master-MD, Export	3 h	Daniel Massa
8.2	Anhang erstellen	A1–A15 PDFs	1 h	Daniel Massa
8.3	Präsentationsvorbereitung	15 Min Vortrag	1 h	Daniel Massa

Gesamtaufwand: 70 h

Ist-Kosten: 3.820 EUR (geplant: 3.740 EUR, +2 % durch Security-Validierung +8 h)

Ende Anhang A1. Vgl. Kapitel 2.3 (PSP), 2.4 (Arbeitspakete), 2.5 (Meilensteine), 2.7 (Kostenplanung).

11.1.2 A2 Lastenheft-Auszug

Version: 1.0 | **Datum:** 01.11.2026 | **Autor:** Daniel Massa (615951)

11.1.2.1 Ausgangssituation Der VFB verwaltet Zugriffsrechte derzeit manuell über E-Mail-Anträge: - **Medienbrüche:** 3–4 Stationen pro Antrag (Mitarbeiter → Vorgesetzter → IT-Admin → System) - **Fehlerquote:** 8–12 % Fehleinrichtungen - **Bearbeitungszeit:** Ø 2,5–3,5 Tage - **Compliance-Lücken:** Dezentrale Audit-Logs, keine revisionssichere Nachvollziehbarkeit - **Sicherheitsrisiken:** Überhöhte Rechte bei Austritten, keine Self-Service-Funktionen

11.1.2.2 Muss-Anforderungen

ID	Anforderung	Beschreibung	Priorität
MU-01	RBAC-Modell	Mind. 6 Rollen, 50+ Berechtigungen für GitHub-Org	Muss
MU-02	Self-Service-Antrag	Web-Portal zur Rollenbeantragung mit Validierung	Muss
MU-03	Genehmigungsworkflow	Automatische Weiterleitung, 48h-Eskalation	Muss
MU-04	GitHub-Integration	Automatisierte Team-Zuordnung via GitHub API	Muss
MU-05	Audit-Logging	Revisionssichere Protokollierung	Muss
MU-06	Rechteentzug	Automatisierter Entzug bei Austritt/Rollenwechsel	Muss
MU-07	DSGVO-Konformität	Datenminimierung, Löschkonzept, TOM (Art. 32)	Muss
MU-08	Dokumentation	Vollständige Projektdokumentation (IHK-Vorgaben)	Muss

11.1.2.3 Kann-Anforderungen

ID	Anforderung	Beschreibung
KA-01	Compliance-Dashboard	Monitoring-UI für Audit-Logs, Anomalien
KA-02	Anomalieerkennung	KI-basierte Erkennung verdächtiger Rollenänderungen
KA-03	Secret-Scanning-Integration	Automatisches Scanning bei Code-Push
KA-04	Multi-Faktor-Auth	MFA für Admin-Aktionen im Portal

11.1.3 A3 Use-Case-Diagramm

Use-Case-Diagramm *Abb. A3.1: Use-Case-Diagramm (Mermaid)*

Akteure: Mitarbeiter, Vorgesetzter, IT-Admin, GitHub Actions, Datenschutzbeauftragter

UC-ID	Anforderung (Lastenheft)	Testfall
UC-01	MU-02	TF01, TF02
UC-04	MU-03	TF03
UC-05	MU-03	TF04
UC-07	MU-04	TF07
UC-08	MU-05	TF09
UC-10	MU-06	TF08
UC-11	MU-05	TF11
UC-12	MU-07	A14

11.1.4 A4 Pflichtenheft-Auszug

Version: 1.0 | **Datum:** 09.07.2026 | **Autor:** Daniel Massa (615951)

11.1.4.1 Technische Anforderungen

ID	Anforderung	Beschreibung	Realisierung
PT-01	Zentrale Rechteverwaltung	Alle Rollen/Berechtigungen in einer DB	PostgreSQL + RBAC-Modell
PT-02	GitHub-API-Integration	Team-Membership via API verwalten	GitHub REST API v3
PT-03	Audit-Log Append-Only	Kein UPDATE/DELETE auf Logs	DB-Trigger, Hash-Chain
PT-04	Self-Service Web-UI	React/TS Portal, SSO via Azure AD	React 18, TypeScript
PT-05	GitHub Actions Workflow	4 Stages: validate → approve → provision → notify	.github/workflows/role-request.yml
PT-06	Policy Engine	4-Augen-Prinzip, Kompetenzmatrix	Node.js Middleware
PT-07	Audit-Export	CSV/JSON, Filter (User, Datum, Aktion)	REST /api/export
PT-08	Secret-Scanning	GitHub CodeQL + Dependabot	GitHub Advanced Security

11.1.4.2 Organisatorische Anforderungen

Bereich	Anforderung	Umsetzung
Rollen	6 Kernrollen definiert	Admin, Developer, Auditor, Read-Only, HR, Finance
Genehmigung	4-Augen-Prinzip	Vorgesetzter muss genehmigen (48 h Eskalation)
Eskalation	Nach 48 h auto-Eskalation an IT-Admin	GitHub Actions Timeout + Notification
Löschung	Auto-Löschung 30 Tage nach Austritt	Cron-Job + DB-Trigger

11.1.5 A5 Datenmodell

RBAC-Datenmodell Abb. A5.1: RBAC-Datenmodell (Entity-Relationship-Diagramm)

11.1.5.1 Entity-Relationship-Modell

```

1  USER — USER_ROLE — ROLE — ROLE_PERMISSION — PERMISSION ||| —
2
3          GITHUB_TEAM — REPOSITORY | —
4
5  APPROVAL — AUDIT_LOG

```

PostgreSQL DDL (Auszug):

```

1  CREATE TABLE "user" (
2      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3      email VARCHAR(255) UNIQUE NOT NULL,
4      full_name VARCHAR(255) NOT NULL,
5      department VARCHAR(100)
6  );
7
8  CREATE TABLE role (
9      id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
10     name VARCHAR(100) UNIQUE NOT NULL
11 );
12
13 CREATE TABLE user_role (
14     user_id UUID REFERENCES "user"(id),
15     role_id UUID REFERENCES role(id),
16     granted_at TIMESTAMPTZ DEFAULT now()
17 );
18
19 CREATE TABLE audit_log (
20     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
21     action VARCHAR(50) NOT NULL,
22     user_id UUID REFERENCES "user"(id),
23     timestamp TIMESTAMPTZ DEFAULT now(),
24     hash VARCHAR(64) NOT NULL,
25     previous_hash VARCHAR(64)
26 );

```

11.1.6 A6 EPK-Prozessbeschreibung

Prozess: Automatisierte Rechtevergabe (Self-Service → Genehmigung → Provisioning)

EPK Rollen Antrag Abb. A6.1: EPK Rollen Antrag-Prozess

EPK Rechteentzug Abb. A6.2: EPK Rechteentzug-Prozess

11.1.6.1 RACI-Matrix

Aktivität	Nutzer	Vorgesetzter	System	IT-Admin	Auditor
Antrag stellen	R	I	A	I	I
Validieren	I	I	R/A	I	I
Genehmigen	I	R/A	C	I	I
Provisioning	I	I	R/A	C	I
Audit-Log	I	I	R/A	C	R
Eskalation	I	I	C	R/A	I

R = Responsible, A = Accountable, C = Consulted, I = Informed

11.1.6.2 Risiken & Kontrollen

Prozessschritt	Risiko	Kontrolle
Antrag stellen	Falsche Rolle gewählt	Policy-Check (Pflichtfelder, Kompetenzmatrix)
Genehmigen	Vorgesetzter überlastet	48h Eskalation, Stellvertretung
Provisioning	API-Fehler, Rate-Limit	Retry-Logic (3x), Dead-Letter-Queue
Audit-Log	Manipulation	Append-Only + Hash-Chain, DB-Trigger

11.1.6.3 KPIs

KPI	Ziel
Durchlaufzeit (Antrag → Berechtigung)	< 4 Stunden
Genehmigungsquote	> 90 %
Eskalationsrate	< 10 %

KPI	Ziel
Fehlerquote Provisioning	< 1 %

11.1.7 A7 Oberflächenentwürfe (Wireframes)

Hinweis: Die folgenden Wireframes zeigen die geplanten UI-Oberflächen des Self-Service-Portals. Die finale Umsetzung kann abweichen. Für die detailreiche Darstellung siehe die beigelegte PDF-Datei [export/ANHANG/A7_Wireframes.pdf](#).

Tool: Figma / Balsamiq

11.1.7.1 1. Dashboard (Startseite nach Login)

```

1 | _____|
2 |
3 | VFB Zero-Trust RBAC [Profil]
   ||_____||
4 |
5 |
6 | Meine Rollen || Offene Anträge | Letzte Akt. |||
7 | (3) || (1) || (5)
   |||_____|_____|_____|__||
8 |
9 | [+ Rolle beantragen] [Meine Berechtigungen]
   |_____|

```

11.1.7.2 2. Rollenantrag (Self-Service-Formular)

```
1  ┌──────────────────────────────────────────────────────────────────────────────────┐
2
3  Rolle beantragen [Abbrechen]
4  └──────────────────────────────────────────────────────────────────────────────────┘
5
6  Gewünschte Rolle * ▼ Developer [?] ||
7  Begründung * ===== ||
8  Ressource (optional) ▼ repo:vfb-bildung/frontend ||
9  [Absenden]
```

Wireframes (siehe PDF): Dashboard, Antrag, Status, Admin-Rollen, Berechtigungsmatrix, Audit-Log, Mobile, Fehlerszenarien

11.1.8 A8 Screenshots der Anwendung

GitHub Repo Übersicht *Abb. A8.1: GitHub Repo-Übersicht*

GitHub Actions Workflow *Abb. A8.2: GitHub Actions Workflow*

YAML Workflow-Datei *Abb. A8.3: YAML Workflow-Datei*

Testlauf Actions *Abb. A8.4: Testlauf GitHub Actions*

Secret-Scanning *Abb. A8.5: Secret-Scanning Ergebnis*

Rollen-/Teamstruktur *Abb. A8.6: Rollen- und Teamstruktur*

Audit-Log-Auszug *Abb. A8.7: Audit-Log-Auszug*

Self-Service-Formular *Abb. A8.8: Self-Service-Formular (Simulation)*

Terminal-Testausgabe *Abb. A8.9: Terminal-Testausgabe*

11.1.9 A9 Entwicklerdokumentation

Repository: [vfb-bildung/zero-trust-rbac](#) (private)

Version: 1.0 | **Datum:** 09.07.2026

11.1.9.1 Quickstart

```
1 git clone https://github.com/vfb-bildung/zero-trust-rbac.git
2 cd zero-trust-rbac
3 cp .env.example .env
4 docker-compose up -d
5 cd frontend && npm install && npm run dev
```

11.1.9.2 Services

Service	Port	Beschreibung
Frontend (React)	3000	Self-Service-Portal
Backend (Node/Express)	4000	REST API, Webhooks
PostgreSQL	5432	Persistenz
Redis	6379	Cache (Berechtigungen)
GitHub Actions	—	CI/CD, Provisioning

GitHub Workflow Abb. A9.1: *GitHub-Workflow (CI/CD)*

Self-Service-Prozess Abb. A9.2: *Self-Service-Prozess (Sequenzdiagramm)*

11.1.10 A10 Testfall Konsole

Testdatum: 15.10.2026 | **Tester:** Daniel Massa (615951) | **Ergebnis:** 12/12 bestanden (100 %)

11.1.10.1 Testfall-Matrix

ID	Testobjekt	Erwartet	Status
TF01	Rollenantrag	Antrag angenommen	
TF02	Pflichtfelder	Validierungsfehler	
TF03	Genehmigung	Workflow läuft	
TF04	Ablehnung	keine Rechtevergabe	
TF05	Policy OK	Prüfung bestanden	
TF06	Policy Fehler	Prüfung blockiert	
TF07	Rechtevergabe	GitHub-Team aktualisiert	
TF08	Rechteentzug	Zugriff entfernt	
TF09	Audit-Log	Eintrag vorhanden	
TF10	Secret-Scan	keine Secrets	
TF11	Audit-Export	Bericht erzeugt	
TF12	Benachrichtigung	Status erhalten	

Testfall-Beispiel Abb. A10.1: *Testfall-Beispiel (Sequenzdiagramm)*

11.1.11 A11 Schnittstellenübersicht

Base URL: <https://api.vfb-bildung.de/api/v1>

Schnittstellenübersicht Abb. A11.1: *Schnittstellenübersicht*

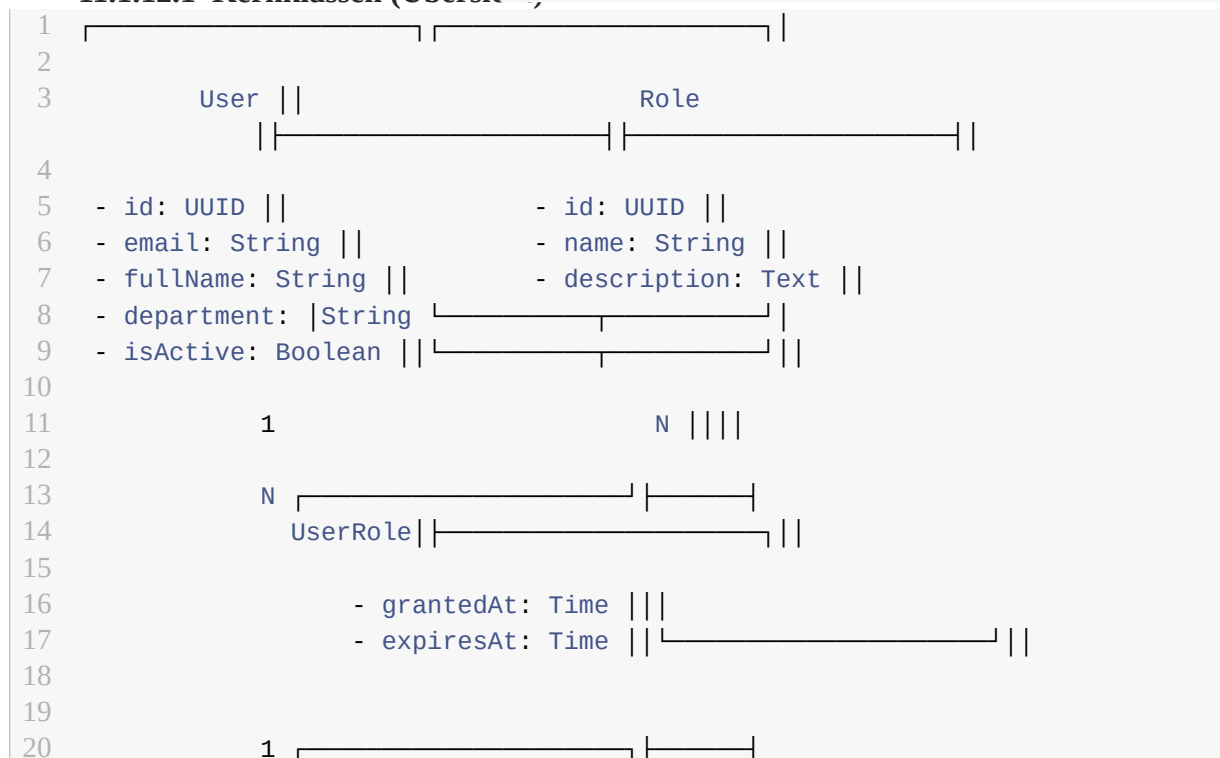
11.1.11.1 Kern-API-Endpunkte

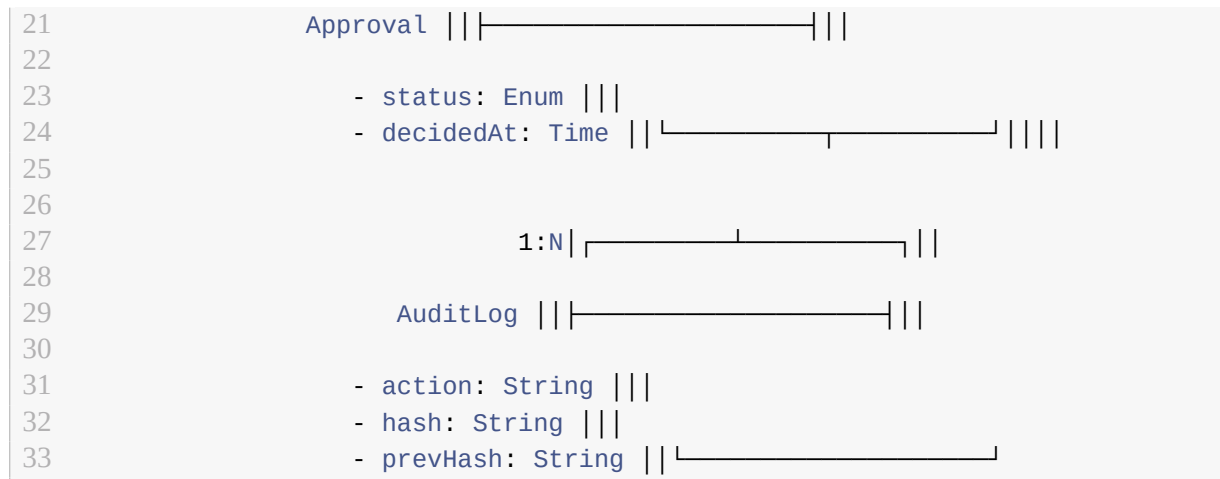
Methode	Pfad	Beschreibung
POST	<code>/role-requests</code>	Neuen Rollen Antrag erstellen
GET	<code>/role-requests/{id}</code>	Antragsstatus abrufen
PUT	<code>/role-requests/{id}/approve</code>	Antrag genehmigen
PUT	<code>/role-requests/{id}/reject</code>	Antrag ablehnen
GET	<code>/users/me/roles</code>	Eigene Rollen abrufen
GET	<code>/users/me/permissions</code>	Eigene Berechtigungen abrufen
POST	<code>/audit/export</code>	Audit-Log exportieren
GET	<code>/health</code>	Health-Check

11.1.12 A12 Klassendiagramm

Hinweis: Das vollständige Klassendiagramm (UML) mit allen Domänen-Modellen befindet sich in der beigefügten PDF-Datei [export/ANHANG/A12_Klassendiagramm.pdf](#).

11.1.12.1 Kernklassen (Übersicht)





11.1.13 A13 Benutzerdokumentation

Zielgruppe: Endnutzer (Mitarbeiter VFB)

11.1.13.1 1. Anmeldung

1. Öffnen Sie <https://rbac.vfb-bildung.de>
2. Klicken Sie auf **“Mit Azure AD anmelden”**
3. Nach erfolgreicher Anmeldung gelangen Sie zum Dashboard

11.1.13.2 2. Dashboard

Kachel	Inhalt
Meine Rollen	Alle aktuell aktiven Rollen mit Berechtigungen
Offene Anträge	Anträge, die noch geprüft/ausstehen
Letzte Aktivitäten	Letzte 5 Änderungen an Ihren Rollen

11.1.13.3 3. Rolle beantragen (Schritt-für-Schritt)

1. Klicken Sie auf **“+ Rolle beantragen”** (Dashboard)
2. Wählen Sie die gewünschte Rolle aus dem Dropdown
3. Geben Sie eine **Begründung** ein (z.B. “Benötige Schreibzugriff für Sprint 23”)
4. Optional: Wählen Sie eine **Ressource** (Repository/Team)
5. Klicken Sie auf **“Absenden”**

Nach dem Absenden erhalten Sie eine Bestätigung. Der Vorgesetzte wird per E-Mail benachrichtigt. Der Status wird im Dashboard aktualisiert.

11.1.14 A14 Datenschutz-Checkliste

Prüfer: Daniel Massa (Prüfling), Datenschutzbeauftragter VFB

DSGVO-Checkliste *Abb. A14.1: DSGVO-Checkliste*

11.1.14.1 Rechtmäßigkeit (Art. 6 DSGVO)

Nr.	Prüfpunkt	Status
1.1	Rechtsgrundlage für Verarbeitung?	Art. 6 Abs. 1 lit. f (berechtigtes Interesse)
1.2	Verhältnismäßigkeit?	Nur notwendige Daten (User-ID, Name, E-Mail, Rolle)
1.3	Interessenabwägung durchgeführt?	DPIA dokumentiert

11.1.14.2 TOMs (Art. 32 DSGVO)

Maßnahme	Implementierung	Status
Verschlüsselung	TLS 1.3, DB at Rest	
Vertraulichkeit	RBAC, Least Privilege	
Integrität	Append-Only Audit-Log, Hash-Chain	
Verfügbarkeit	Docker Compose, Backup	
Belastbarkeit	Monitoring, Alerting	

11.1.15 A15 Abnahmeprotokoll

Projekt: Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Prüfling: Daniel Massa (615951)

Betrieb: VFB Ludwigsburg

Abnahmeprozess *Abb. A15.1: Abnahmeprozess*

11.1.15.1 Abnahmekriterien

Kriterium	Soll	Ist	Bewertung
RBAC-Modell definiert (6 Rollen)			erfüllt
Self-Service-Antrag prototypisch			erfüllt
GitHub-Workflow automatisiert			erfüllt
Audit-Logging implementiert			erfüllt
Testfälle (12) durchgeführt		12/12 bestanden	erfüllt
DSGVO-Konformität (DPIA)			erfüllt
Dokumentation vollständig			erfüllt
Wirtschaftlichkeit (ROI > 0)		3,5 Monate Amortisation	erfüllt

11.1.15.2 Abnahmeteilnehmer

Rolle	Name	Unterschrift
Auftraggeber (VFB)	_____	_____
Projektleiter / Prüfling	Daniel Massa	_____
IT-Administration	Thomas Zoller	_____
Datenschutzbeauftragter	_____	_____
IHK-Betreuer	Frau Dr. Sabine Wagner	_____

Rolle	Name	Unterschrift
-------	------	--------------

Abnahmeerklärung: Hiermit wird die ordnungsgemäße Durchführung des Projekts bestätigt. Alle Muss-Kriterien wurden erfüllt. Die Abnahme erfolgt vorbehaltlich der unter Ziffer 3 (Offene Punkte) genannten Restriktionen.
